

Correct Code, Vulnerable Dependencies: A Large Scale Measurement Study of LLM-Specified Library Versions

CHENGJIE WANG^{*†}, JINGZHENG WU^{*‡}, XIANG LING^{*‡}, TIANYUE LUO^{*}, and CHEN ZHAO^{*}

Large language models (LLMs) are now largely involved in software development workflows, and the code they generate routinely includes third-party library (TPL) imports annotated with specific version identifiers. These version choices can carry security and compatibility risks, yet they have not been systematically studied.

We present the first large-scale measurement study of version-level risk in LLM-generated Python code, evaluating 10 LLMs on PinTrace, a curated benchmark of 1,000 Stack Overflow programming tasks. LLMs tend to specify version identifiers when directly prompted at 26.83%–95.18%, while down to 6.45%–59.19% in creating a manifest file directly. Among the specified versions, 36.70%–55.70% of tasks contain at least one known CVE, and 62.75%–74.51% of them carry Critical or High severity ratings. In 72.27%–91.37% of cases, the associated CVEs were publicly disclosed before the model’s knowledge cutoff. The statistics show all models converge on the same small set of risky release versions, indicating a systemic bias rather than isolated model error. Static compatibility rates range from 19.70% to 63.20%, with installation failure as the dominant cause. The dynamic test cases confirm the pattern by 6.49%–48.62% pass rates. Further experiments confirm that these failures are attributable to version selection rather than code quality, and that externally anchored version constraints substantially reduce both vulnerability exposure and compatibility failures. Our findings reveal LLM version selection as a first-class, previously overlooked risk surface in LLM-based development. We disclosed these findings to the community of the evaluated models, and several confirmed the issue. All the code and dataset have been released for open science at <https://github.com/dw763j/PinTrace>.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Risk management**; *Software libraries and repositories*.

Additional Key Words and Phrases: large language models, version specify, software supply chain security

ACM Reference Format:

Chengjie Wang, Jingzheng Wu, Xiang Ling, Tianyue Luo, and Chen Zhao. 2026. Correct Code, Vulnerable Dependencies: A Large Scale Measurement Study of LLM-Specified Library Versions. 1, 1 (May 2026), 35 pages. <https://doi.org/xxxx/xxxx>

1 Introduction

Large language models (LLMs) have become a central fixture of modern software development. GitHub Copilot alone surpassed 20 million users by mid-2025, with deployment across 90% of Fortune 100 companies [12]. Copilot now contributes an average of 46% of all code to GitHub [11], and roughly 80% of new developers on GitHub adopt it within their first week [12]. AI coding

^{*}Intelligent Software Research Center, Institute of Software, Chinese Academy of Sciences, Beijing, China

[†]University of Chinese Academy of Sciences, Beijing, China

[‡]Key Laboratory of System Software (Chinese Academy of Sciences), Beijing, China

Authors’ Contact Information: [Chengjie Wang](#), chengjie2021@iscas.ac.cn; [Jingzheng Wu](#), jingzheng08@iscas.ac.cn; [Xiang Ling](#), lingxiang@iscas.ac.cn; [Tianyue Luo](#), tianyue@iscas.ac.cn; [Chen Zhao](#), zhaochen@iscas.ac.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/5-ART

<https://doi.org/xxxx/xxxx>

assistance has moved from an optional productivity tool to an expected part of the developer workflow [16, 73], and with that shift comes new, largely unaudited risks in the code being produced.

The development of modern software projects depends on third-party libraries (TPLs). Studies of large open-source ecosystems document pervasive TPL adoption, with the transitive dependency graph of a typical application spanning dozens to hundreds of packages [48, 54]. In traditional development, dependency version management is a deliberate, human-controlled process in which developers consult changelogs, security advisories, and compatibility notes before committing a specific release to a manifest file [22, 47]. When an LLM generates code, however, those same version choices are made implicitly and at scale, folded into the generated output without the developer explicitly deciding what version to use. A pinned version may carry known Common Vulnerabilities and Exposures (CVE), silently exposing every project that adopts the generated code to catalogued security risks [38]. Equally, a pinned version may be incompatible with the generated code's API usage, rendering the snippet non-executable despite appearing syntactically correct [18, 24]. Both failure modes are invisible to a developer who adopts LLM-generated code without independent dependency auditing.

Prior work has examined two related but orthogonal risks in LLM-generated code, yet neither addresses the version dimension. One line of research measures whether the *logic* of generated code contains security weaknesses, finding that LLMs produce snippets with Common Weakness Enumeration (CWE)-classified vulnerabilities at rates estimated between 12% and 65% across models and task configurations [5, 8, 10, 53, 77]. A second line documents that LLMs hallucinate non-existent package names at non-trivial rates, creating a supply-chain attack surface through which malicious actors can register the hallucinated names [30, 32, 33, 59]. Neither line of work addresses the *version* dimension: whether the specific release of an existing, correctly named library that an LLM recommends is safe to use and compatible with the generated code. This gap matters because version-level risks are structural. If LLMs converge on a small set of popular but vulnerable versions, then the same CVE propagates silently and uniformly into every project that adopts LLM-generated code referencing that library [38], regardless of what task the code was written for.

We present the first large-scale measurement study of LLM version selection behavior. We focus on the Python ecosystem because LLM-assisted Python development is pervasive [62] and PyPI's rich vulnerability and release metadata makes version-level risk measurable [51]. We evaluate 10 LLMs on PinTrace, a curated benchmark of real-world Stack Overflow programming tasks, examining three dimensions: version specification behavior, security vulnerability exposure, and static and dynamic compatibility.

Our analysis of LLM-generated code on PinTrace reveals systematic security and compatibility risks consistent across all ten evaluated models. First, version-annotation behavior is governed by format affordance rather than consistent engineering intent. Under inline-comment prompting (*inline* mode), LLMs specify versions for 26.83%–95.18% of library references. Under manifest-based prompting (*explicit* mode), the same models specify only 6.45%–59.19% of references. Across both modes, version choices concentrate on a narrow band of popular releases that lags each model's knowledge cutoff by 9 to 31 months. Second, the security consequences are severe. Among tasks where LLMs specify versions, 36.70%–55.70% contain at least one version carrying a known CVE, with 62.75%–74.51% of those vulnerable versions rated *Critical* or *High* severity. In 72%–91% of cases, the associated CVEs were publicly disclosed before each model's knowledge cutoff. Yet the same risky versions appear consistently across all model families, pointing to a shared systemic bias rather than an isolated model defect. Third, the specified versions are frequently incompatible with the generated code. Static compatibility rates fall to 19.70%–63.20% under inline prompting. Dynamic verification confirms the pattern, with pass rates collapsing to 6.49%–48.62% as version

incompatibilities block execution before any test can run. Together, these three findings establish that LLM version selection introduces risks along both the security and compatibility dimensions.

We conducted three additional analyses to characterize the boundaries and root causes of these risks. We first examine whether the compatibility findings hold across different Python runtime environments. Compatibility rates peak at mid-range Python versions and collapse sharply at more recent releases, with representative models dropping by more than 50 percentage points across that transition. Under explicit mode the trend reverses, with compatibility improving monotonically toward newer Python versions. This reversal confirms that LLMs systematically prefer older library releases that are compatible with legacy environments but fail under modern runtime configurations. A controlled diagnosis experiment then establishes that installation failures are attributable to version selection rather than to defects in the generated code itself. A mitigation probe shows that natural-language safety instructions yield no meaningful improvement, while external version anchoring substantially reduces vulnerability exposure. RAG-augmented grounding provides marginal additional compatibility gains, though a residual gap remains. These analyses confirm that version-selection risk is addressable only through external tooling, not through model-level prompting. We disclosed these findings to the community of all evaluated models and to major coding assistant providers, several of whom have acknowledged the problem.

The contributions of this paper are as follows:

- A measurement study of LLM version selection behavior across ten models on our curated PinTrace dataset with 1,000 real-world Stack Overflow questions, covering version specification rates, validity, vulnerability exposure, and compatibility.
- Empirical evidence that LLM-specified versions are systematically vulnerable, converge across model families on the same small set of risky releases, and lag behind each model's knowledge cutoff due to a systemic training-signal bias.
- A diagnosis experiment establishing that installation-level compatibility failures are attributable to version selection rather than generated code quality, and a mitigation probe characterizing the effectiveness and limits of prompt-level interventions.
- A dataset and evaluation pipeline for reproducible version-level security and compatibility assessment of LLM-generated code, released at <https://github.com/dw763j/PinTrace>.

The remainder of the paper is organized as follows. §2 provides background on LLM-assisted development and software supply chain security. §3 describes the study design. §4 details the experimental setup. §5 presents the main results for RQ1–RQ3. §6 reports the robustness, diagnosis, and mitigation analyses. §7 discusses implications for developers and LLM providers. §8 addresses threats to validity. §9 surveys related work. §10 concludes.

2 Background

This section provides the prior knowledge needed to understand our study: how LLMs have changed the dependency annotation process, and why version-level choices represent an underexplored point of failure in software supply chain security.

2.1 LLM-Assisted Development and Dependency Annotation

LLMs have fundamentally changed how software is written [19]. Tools such as GitHub Copilot, Cursor, and Claude Code are now embedded into the daily workflow of millions of developers [75]. The 2025 Stack Overflow Developer Survey reports that 84% of developers use or plan to use AI coding tools, and 51% of professionals do so daily [62]. These tools do not merely suggest individual lines. They generate complete, self-contained code snippets that include import statements, function bodies, and dependency annotations in a single pass [12, 26, 36]. As a result, version choices for

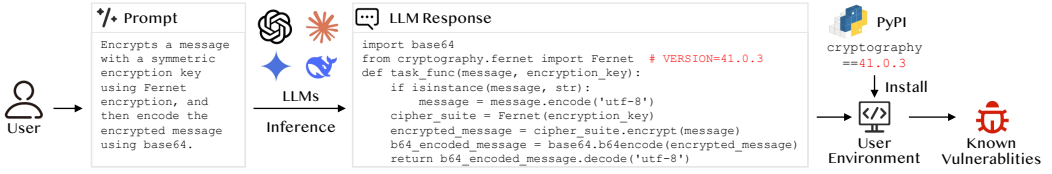


Fig. 1. A motivating example of LLM-introduced version-level risk. An LLM generates functionally correct code and annotates the import with `cryptophy==41.0.3`. The version installs successfully, yet carries 7 known CVEs, silently exposing the user's environment to catalogued vulnerabilities.

TPLs now flow into codebases through an automated, largely unreviewed channel, at a scale and speed that traditional human-driven dependency management was not designed to handle.

When developers write code manually, selecting a library version is a deliberate act where they consult changelogs, security advisories, and compatibility notes before committing a specific release to a manifest file [22, 47]. LLMs displace this decision with parametric inference. The model draws on version information encoded in its training corpus, without access to live vulnerability databases or the ability to verify whether the chosen release is current, safe, or installable [31, 32]. A developer who accepts a generated snippet inherits the model's version choice wholesale, often without realizing a choice was made at all. The scale of this risk has begun to surface in industry data: analysis of 36,870 LLM-assisted dependency upgrade recommendations found that 27.76% of version suggestions were hallucinations [58]. Some recommendations pointed to packages that were outright malware. Figure 1 illustrates a concrete instance of this problem from the BigCodeBench dataset [79]. Given a task that requires the `cryptophy` library, an LLM produces syntactically correct, functionally plausible code and annotates the import with `# VERSION=41.0.3`. The version exists on PyPI and installs without error, so neither the developer nor a build system raises an alert. Yet `cryptophy==41.0.3` carries 7 known CVEs, exposing the user's environment to catalogued vulnerabilities without any visible warning. This scenario, invisible in the absence of explicit security auditing, is the central concern of this paper.

2.2 Software Supply Chain Security and the Version Control Point

Third-party libraries substantially improve development efficiency in the form of software supply chain [44, 57], but they also expand the attack surface of every project that uses them [9, 30, 54, 55, 69]. At the scale of modern registries, vulnerability exposure is not an edge case: in 2025, one in five PyPI releases was associated with a vulnerability rated CVSS 7.0 or higher [58]. High-profile incidents, including Log4Shell [45], SolarWinds [1], and the XZ Utils backdoor [49], demonstrate that a single compromised or vulnerable version in a widely-used library can trigger systemic compromise at a global scale. Once a vulnerability is present in a widely-used library version, every project depending on that version inherits the same exposure through the dependency graph, with no per-project variation [6, 38, 72].

To manage these risks, the software engineering community has developed software composition analysis (SCA) tools and software bill of materials (SBOM) frameworks that identify and track third-party components and their known vulnerabilities [20, 21, 48, 63, 68]. These approaches are effective at auditing committed dependency files and flagging vulnerable versions after the fact. However, SCA and SBOM operate post hoc. By the time the tools run, the version has already been written into the manifest, and the window for intervention has closed. When that manifest is generated by an LLM, the version choice is made at generation time, before any developer review or security tooling is involved. This paper addresses that upstream gap: not whether a vulnerable version is eventually detected, but how reliably LLMs introduce one in the first place.

3 Study Design

Though LLMs introduce version choices into codebases implicitly, those choices are not audited at the point of generation. To characterize this risk empirically, we design a measurement study organized around three research questions. §3.1 defines the questions and the rationale connecting them. §3.2 describes how we construct the task dataset. §3.3 introduces the two prompting modes. §3.4 presents the analysis pipeline. §3.5 formally defines all evaluation metrics.

3.1 Research Questions

We organize the study around three research questions that together characterize version-level risk from its origin to its consequences. RQ1 establishes the empirical baseline: what versions LLMs actually produce and whether those identifiers are valid. RQ2 probes the security consequences of those choices. RQ3 asks whether the chosen versions are executable in practice.

RQ1. Version specification behavior: *When generating code, do LLMs annotate TPL dependencies with version identifiers, and are the specified versions valid published releases?* Unpinned dependencies introduce non-reproducible builds and silent upgrade risks irrespective of any known vulnerability [15, 29]. Invalid versions cannot be installed and reveal the model’s tendency to hallucinate plausible but nonexistent identifiers. RQ1 therefore provides the empirical baseline on which RQ2 and RQ3 are built.

RQ2. Version vulnerability exposure: *Among the valid versions that LLMs specify, what proportion carry known CVE vulnerabilities and how are they distributed across severity levels?* Prior work has measured whether generated code contains insecure logic [5, 8, 53, 77] or whether recommended packages exist [30, 32, 33, 59]. Our focus is orthogonal, asking whether the specific versions of real, correctly named libraries expose users to catalogued vulnerabilities. A high exposure rate signals a systemic risk, because the same CVE propagates into every project that adopts LLM-generated code referencing that library.

RQ3. Version-Code compatibility: *Are the TPL versions that LLMs specify compatible with the generated code?* Even a valid, vulnerability-free version may be incompatible with the API usage in the generated code, making the snippet non-executable [40, 74, 80]. We decompose compatibility into two dimensions: *static compatibility*, assessed via installation success and static type checking, and *dynamic compatibility*, assessed via execution of test suites against the TPL versions.

3.2 Dataset Construction

To ground the study in authentic development scenarios, we build a task dataset from real-world programming Q&A rather than synthetic benchmarks.

Programming language. Rigorous vulnerability analysis and compatibility verification require a controlled execution environment, a comprehensive vulnerability database, and a reproducible package ecosystem. These constraints are best satisfied by committing to a single programming language. We focus on Python for three reasons. First, Python is the dominant language in data science, machine learning, and scripting, the domains most frequently targeted by LLM-assisted coding [12, 62]. LLM dependency recommendations therefore carry the greatest practical consequence in Python projects. Second, PyPI hosts over 600,000 packages with comprehensive release histories and vulnerability records [58]. This breadth provides the infrastructure needed for version validation and vulnerability lookup at scale. Third, PyPI’s rapid release cycles, reflected in a 50.64% year-over-year growth in downloads in 2025 [58], produce version-level compatibility failures at a frequency that makes Python a high-signal environment for the phenomena we study.

Data source. We derive tasks from Stack Overflow, a website of Q&A about programming. Stack Overflow questions reflect genuine development scenarios encountered by real practitioners,

and their accepted answers represent community-validated solutions that provide a meaningful reference for our analysis.

Selection criteria. We filter the dump by five criteria:

- The question has an accepted answer.
- The accepted answer contains at least one fenced code block.
- The code block parses successfully as a Python abstract syntax tree (AST), excluding pseudocode, shell scripts, and malformed snippets.
- The parsed code contains at least one `import` statement referencing a third-party library.
- The question was posted between January 1, 2020 and January 1, 2026.

The temporal window spans three distinct knowledge regimes. Questions from 2020–2022, predating widespread LLM adoption, are likely to have entered LLM training corpora, providing tasks on which models may recall specific version information. Questions from 2022–2024 coincide with active LLM deployment. Questions from 2025 are likely beyond the knowledge cutoff of most evaluated models, probing version knowledge under distribution shift. This spread allows us to examine whether version-annotation accuracy varies across these regimes.

Balanced sampling. The TPLs referenced in Stack Overflow question-answer pairs are not uniformly distributed: commonly used libraries such as `pandas` and `numpy` may dominate the raw question volume, and naive random sampling would over-represent this small set of high-volume libraries, reducing the ecological diversity of the study. We therefore apply TPL-balanced sampling. We restrict to posts with non-empty target libraries, let $\mathcal{K}$ denotes the number of distinct libraries in the pool, and draw N posts by assigning each library a target count of approximately $N/\mathcal{K}$ and greedily selecting disjoint posts to satisfy these targets where possible, then filling any shortfall by choosing posts that most reduce under-coverage of their mentioned libraries. The resulting subset is not proportional to usage frequency in the raw data, but is intended to give each library a comparable chance of inclusion and to improve ecological diversity.

3.3 Prompting Modes

To ensure measurement results are representative and cross-model comparisons are valid, we standardize two prompting modes that correspond to the two common ways developers request dependency-annotated code from an LLM. Using controlled modes rather than free-form prompts relieves prompt-wording variance as a confound and makes results comparable across models.

Explicit mode. The prompt instructs the LLM to generate both a code snippet and a companion `requirements.txt` file listing each dependency with a version specifier. This reflects standard software-engineering practice and supports relational specifiers, *e.g.*, `==`, `>=`, `~=`, where models may express version ranges rather than exact pins.

Inline mode. The prompt instructs the LLM to annotate each third-party import with a version identifier in a trailing comment, following the convention `import lib # version==x.y.z`. This mode reflects the practice of sharing self-contained snippets in chat sessions, Jupyter notebooks, or single-file scripts, where no manifest is expected.

The two modes are complementary rather than redundant. Explicit mode captures how developers request structured dependency management. Inline mode captures the embedded version annotation that accompanies everyday code generation. Together, they cover the full range of version-specification behavior that practitioners encounter. The full prompts are appended based on the basic structure of the prompt from the BigCodeBench dataset [79].

3.4 Analysis Pipeline

We analyze version-level security and compatibility through a five-stage pipeline, illustrated in Figure 2. Each task $t \in \mathcal{T}$ is processed under every combination of model $m \in \mathcal{M}$ and prompting

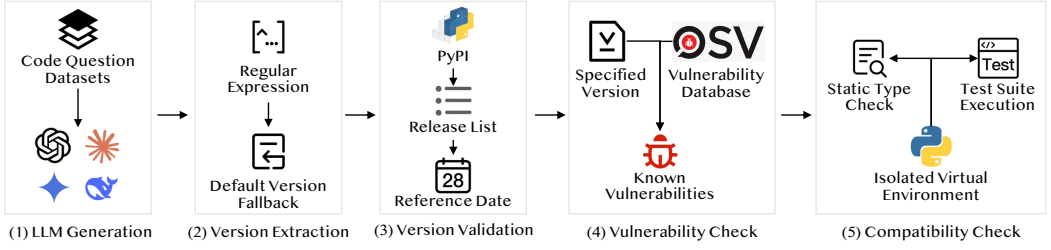


Fig. 2. The analysis pipeline of the measurement study. Each (task, model, mode) triple passes through code generation, version extraction, version validation, vulnerability lookup, and compatibility verification. Stages (1)–(4) supply data for RQ1 and RQ2. Stage (5) supplies data for RQ3.

mode $p \in \mathcal{P} = \{\text{inline}, \text{explicit}\}$, yielding the triple (t, m, p) as the basic unit of analysis. Stages (1)–(4) produce the data for RQ1 and RQ2. Stage (5) addresses RQ3.

(1) LLM generation. We submit question text q_t to model m under prompting mode p and record the raw output $G(t, m, p)$.

(2) Version extraction. We parse $G(t, m, p)$ to identify all TPL references and their associated version specifiers, yielding a set of (library, version) pairs $\mathcal{V}(t, m, p) = \{(\ell_1, v_1), \dots, (\ell_k, v_k)\}$. Under inline mode, we apply a regular expression over import-comment patterns. Under explicit mode, we parse the requirements.txt block line by line. For libraries without any version specifier, we resolve the version to the latest PyPI release as of a unified cutoff date for all models.

(3) Version validation. For each $(\ell, v) \in \mathcal{V}(t, m, p)$, we query the PyPI JSON API to verify that version v is a published distribution of library ℓ . Versions are marked invalid if they are absent from PyPI’s release history, have been yanked, or belong to packages unavailable on PyPI.

(4) Vulnerability check. For each valid (ℓ, v) , we query a pre-built vulnerability index to retrieve all CVE records affecting that specific version, along with their Common Vulnerability Scoring System (CVSS) severity classifications. The construction of this index from the Open Source Vulnerability (OSV) dataset [13] is described in §4.

(5) Compatibility check. We assess compatibility in two sequential steps that together cover both static and dynamic dimensions.

Static type check. For each task t , we create an isolated Python virtual environment and attempt to install all TPLs in $\mathcal{V}(t, m, p)$ simultaneously. For tasks where installation succeeds, we run a static type checker on $G(t, m, p)$ against the installed library stubs, detecting API-level incompatibilities such as removed functions or changed signatures. A task is statically compatible if installation succeeds and the type checker reports no errors.

Dynamic execution check. Static type compatibility is a necessary but not sufficient condition for runtime correctness. To obtain execution-level evidence, we integrate our pipeline with BigCodeBench [79], an established benchmark that pairs Python programming tasks with ground-truth test suites spanning a range of TPLs. This integration enables static and dynamic signals to be collected on the same tasks under identical environments, providing a complementary and cross-validating view of version compatibility. For each BigCodeBench task, we construct the same isolated environment as above and execute the provided test suite. A task passes if all suite assertions hold.

Taken together, the pipeline transforms each (t, m, p) triple into a structured record covering version specification, validity, vulnerability exposure, and compatibility, providing the empirical data for all three research questions.

3.5 Metrics

We formally define the main metrics used in our evaluations. Let $L(t, m, p)$ denote the multiset of all third-party library references in $G(t, m, p)$. We define three subsets of L . $L_S \subseteq L$ contains library uses that carry an explicit version specifier. $L_V \subseteq L$ contains library uses whose resolved version corresponds to a published PyPI release, where the resolved version is either the explicitly specified version or the latest-release fallback assigned in Stage (2). $L_U \subseteq L_V$ further restricts to versions carrying at least one associated CVE. The filtering applied by Stages (2)–(4) determines membership in L_S , L_V , and L_U , respectively.

Library-level rates measure behavior aggregated over all library uses across tasks. Three rates share the same form, each measuring the fraction of library uses satisfying a given criterion relative to all library uses L :

Metric	Symbol	Ratio
Version specification rate	ρ_S	$ L_S / L $
Version validity rate	ρ_V	$ L_V / L $
Library vulnerability rate	ρ_U	$ L_U / L_V $

where all sums are taken over $t \in \mathcal{T}$ for a given (m, p) .

High-risk rate (ρ_H) characterizes the severity profile of vulnerable versions by measuring what fraction are rated *Critical* or *High* by CVSS:

$$\rho_H(m, p) = \frac{|\{(\ell, v) \in \bigcup_t L_U(t, m, p) : \text{sev}(\ell, v) \in \{\text{Critical}, \text{High}\}\}|}{|\bigcup_t L_U(t, m, p)|} \quad (1)$$

Task-level metrics measure risk at the granularity of individual tasks rather than individual library uses. Task vulnerability exposure (τ_U) counts the fraction of tasks in which at least one specified version carries a CVE:

$$\tau_U(m, p) = \frac{|\{t \in \mathcal{T} : L_U(t, m, p) \neq \emptyset\}|}{|\mathcal{T}|} \quad (2)$$

This captures user-facing risk more directly than ρ_U , since a single vulnerable library is sufficient to expose the entire task.

Task compatibility rate (τ_C) is the fraction of tasks that pass Stage (5):

$$\tau_C(m, p) = \frac{|\{t \in \mathcal{T} : \text{compat}(t, m, p) = 1\}|}{|\mathcal{T}|} \quad (3)$$

The same formula applies to both static and dynamic compatibility. The specific task population in each case is clarified in the corresponding results section.

4 Experiment Setup

This section documents the concrete configuration of the study. §4.1 reports the dataset statistics. §4.2 lists the evaluated LLMs and their inference settings. §4.3 describes the construction of the vulnerability index. §4.4 specifies the execution environment for compatibility verification.

4.1 Dataset Statistics

PinTrace is constructed from the official Stack Overflow data dump [60], which archives all posts and answers on the website, and it is updated on January 6, 2026. The full dump contains 24,178,621 questions, of which 12,392,896 carry an accepted answer.

Before applying the selection criteria, we first establish the library boundary for the study. We restrict the candidate pool to libraries that appear in the OSV PyPI vulnerability database [13],

Table 1. Temporal distribution of PinTrace dataset. The three knowledge regimes reflect each year’s likely relationship to LLM training corpora: pre-adoption questions are likely memorized, deployment-era and near-cutoff questions may be partially covered, post-cutoff questions probe out-of-distribution version knowledge.

Year	# Tasks	Cumulating (%)	# Unique Libs	Knowledge Regime
2020	313	31.3	177	Pre-LLM adoption
2021	246	55.9	153	Pre-LLM adoption
2022	194	75.3	133	Active LLM deployment
2023	131	88.4	101	Active LLM deployment
2024	76	96.0	74	Near-cutoff
2025	40	100.0	49	Post-cutoff (most models)
Total	1,000	100.0	267	—

i.e., libraries for which at least one CVE record exists in the index. This boundary ensures that every task in the dataset is capable of producing a meaningful vulnerability signal in Stage (4), and it aligns the library scope of the dataset with the scope of the vulnerability index, eliminating a systematic blind spot in which the dataset contains libraries that the index cannot evaluate.

Within this boundary, applying the selection criteria in §3.2 yields 405,707 candidate records after filtering for Python context and third-party library involvement. TPL-balanced sampling then reduces this pool to a final dataset of **1,000 tasks** covering **267 distinct third-party libraries**.

Python’s import system allows a package to be imported under a name that differs from its registered name on PyPI. For example, the PIL import corresponds to the pillow package on PyPI, and sklearn corresponds to scikit-learn. To address this, we apply the widely adopted import-to-package name mapping included in the pipreqs project [4], which covers 1,157 widely used Python packages exhibiting such discrepancies and is actively maintained by the community.

Table 1 reports the temporal and library distribution of tasks. The dataset is front-weighted toward 2020–2022, reflecting the higher volume of Stack Overflow activity in that period. Tasks from 2024 onward are fewer, partly because later questions are more likely to fall beyond the knowledge cutoff of the evaluated models, and partly because the rise of LLM-based coding assistants has reduced the volume of questions posted to Stack Overflow from over 182,948 questions per month in 2020 to a maximum of 18,239 a month in 2025, which is roughly 10% remained [61]. The dataset nonetheless retains 116 tasks from 2024–2025 to probe version knowledge under distribution shift. Unique library counts per year are broadly proportional to task counts, indicating that the temporal front-weighting does not artificially concentrate the library coverage in any single period.

The 267 libraries span a wide range of application domains, including web frameworks (*e.g.*, django, flask, fastapi), data science and machine learning (*e.g.*, pandas, numpy, torch), security and cryptography (*e.g.*, cryptography, pycryptodome), and network utilities (*e.g.*, requests, paramiko, scrapy). The five most frequent libraries are pandas (142 tasks), numpy (123), django (88), flask (61), and requests (56). While the remaining 262 libraries each appear in at most 26 tasks, confirming that balanced sampling prevents any single library from dominating the dataset.

To understand the PinTrace dataset, we summarize the content characteristics of the dataset in Table 2. On the question side, the median token length is 286.5 tokens, but the high standard deviation of 518.3 tokens reflects the wide range of question complexity in Stack Overflow posts, from brief one-line queries to detailed multi-paragraph problem descriptions. On the answer side, accepted code blocks have a median of 16 lines and 123.5 tokens, confirming that the tasks consist of self-contained, executable snippets rather than skeletal stubs or pseudocode. The mean TPL import count of 1.6 per task indicates that most tasks involve at least one third-party library, with a non-trivial fraction importing two or more. Specifically, 46.4% tasks import at least two distinct

Table 2. Content statistics of the PinTrace dataset. Token counts are calculated by the GPT-5 tokenizer from the ChatGPT official tool tiktoken [46]. TPL import count is measured on the accepted answer’s code block. The standard deviation (Std.) is computed across all 1,000 tasks.

Statistic	Mean	Median	Std.
<i>Question</i>			
Token length	430.6	286.5	518.3
<i>Accepted answer code block</i>			
Lines of code	24.9	16.0	39.3
Token length	202.0	123.5	298.8
TPL imports per task	1.6	1.0	0.8
<i>Library coverage</i>			
Distinct TPLs			267
Tasks with ≥ 2 TPL imports		464 (46.4%)	
Tasks with ≥ 3 TPL imports		131 (13.1%)	

libraries and 13.1% tasks import at least three, providing sufficient multi-dependency structure to observe the version-conflict and concurrent-vulnerability patterns central to RQ2 and RQ3.

4.2 LLMs Under Evaluation

The ten LLMs are selected to vary four dimensions simultaneously: deployment prevalence, model scale, knowledge cutoff date, and intra-vendor generational diversity. Table 3 lists all ten models we evaluated with their basic properties. Deployment prevalence is represented by the three most widely used commercial coding assistants (GPT-5.4, Claude-Sonnet-4.6, Gemini-3.1-Pro) alongside seven open-source alternatives. Model scale ranges from 30B to 1T parameters. Knowledge cutoff spans April 2024 to January 2026. Intra-vendor diversity is covered by including both Qwen3 and Qwen3.5 from Alibaba to compare successive generations, and Qwen3-235B alongside Qwen3-30B to compare scales within the same generation. This design enables us to examine whether version-annotation behavior is driven by model capability, training recency, or weight availability, rather than by any single model family.

All models are queried through a unified API under the same calling protocol. Each query is issued with a timeout of 120 seconds and retried automatically on transient failures to prevent missing data. The sampling temperature is set to the provider’s default to reflect realistic code generation conditions.

4.3 Vulnerability Index

Stage (4) of the analysis pipeline requires a pre-built mapping from (ℓ_k, v_k) pairs to known vulnerability records. We construct this index offline from the OSV dataset [13], a community-maintained vulnerability database covering the PyPI ecosystem with structured records.

The construction proceeds in three steps. First, we download all OSV JSON records and retain those whose ecosystem field is PyPI. Second, for each retained record, we resolve the affected version set from two sources: explicit version lists in the record, and range specifications encoding introduced, fixed, and last_affected boundaries. Range specifications are expanded into concrete version enumerations by cross-referencing the full PyPI release history of each package. Where a single OSV record carries multiple aliases, we deduplicate to the canonical CVE identifier before indexing. Finally, we build the index mapping each (ℓ_k, v_k) pair to its canonical CVE identifiers and their CVSS severity scores for the vulnerability check.

Table 3. LLMs evaluated in this study. “Closed” and “Open” refer to model-weight availability. For MoE models, active/total parameters are reported.

Model	Type	Params	Release	Cutoff
GPT-5.4	Closed	–	2026-03-05	2025-08
Claude-Sonnet-4.6	Closed	–	2026-02-17	2025-05
Gemini-3.1-Pro [†]	Closed	–	2026-02-19	2025-01
DeepSeek-V3.2	Open	37B/671B	2025-12-01	2025-05 [‡]
Kimi-K2.5	Open	32B/1T	2026-01-27	2024-04 [‡]
Qwen3.5-397B [†]	Open	17B/397B	2026-02-15	2026-01 [‡]
Qwen3-235B [†]	Open	22B/235B	2025-07-21	2024-12 [‡]
Qwen3-30B [†]	Open	3B/30B	2025-07-21	2024-12 [‡]
MiniMax-M2.5	Open	10B/229B	2026-02-12	2024-06 [‡]
Llama-4-Scout	Open	17B/109B	2025-04-05	2024-08

[†] Abbreviated from their full names: Gemini-3.1-Pro-Preview, Qwen3.5-397B-A17B, Qwen3-235B-A22B-Instruct-2507, and Qwen3-30B-A3B-Instruct-2507.

[‡] Knowledge cutoff dates inferred by querying the model directly, as the provider does not officially publish them.

Both the OSV snapshot and the PyPI release metadata are anchored to February 2026, ensuring a consistent vulnerability knowledge base across all pipeline runs.

4.4 Experiment Environment

All experiments run on a virtual server with a 16-core AMD EPYC 7713 CPU, 64 GB RAM, and 1.5 TB storage, running Ubuntu 24.04 LTS.

Version fallback. For library references without an explicit version specifier, we resolve the version to the latest PyPI release available as of February 2026, the unified temporal anchor for all models and pipeline runs as discussed in Stage (2) of §3.4.

Environment isolation. Each $G(t, m, p)$ triple is evaluated in a fully isolated temporary Python virtual environment managed by `uv@0.10.7` [2], which resolves and installs all dependencies from scratch per run to prevent cross-task package contamination. We use Python 3.12 as the primary runtime, which was released in October 2023 [52] and falls within the cutoff of evaluated models.

Static type checker. We use `ty@0.0.23` [3] to assess static compatibility. `ty` resolves type information directly from the packages installed in the `uv` environment, so it reflects the exact API surface of the specified versions. We invoke `ty` with its default configuration and treat any reported diagnostic as an incompatibility signal.

BigCodeBench subset. BigCodeBench [79] is a large-scale code generation benchmark constructed in Python, containing instruction-following prompts, reference solution code, and test suites. Of BigCodeBench’s 1,140 tasks, 813 contain at least one third-party library import. We further restrict to the 724 tasks where at least one imported library appears in the vulnerability index, applying the same OSV-membership criterion used for PinTrace, so that security and compatibility analyses can be applied consistently alongside dynamic execution. Each task executes within the same isolated `uv` environment as Stage (5), with a per-task timeout of 180 seconds.

5 Results

This section presents the empirical findings of all three research questions. §5.1 characterizes whether and how LLMs annotate dependencies with version identifiers, and what those identifiers look like (RQ1). §5.2 measures the security consequences of those version choices (RQ2). §5.3 evaluates whether the chosen versions are compatible with the generated code (RQ3).

Table 4. Version specification behavior and vulnerability exposure across models and prompting modes. #LibUses denotes total third-party library mentions across all tasks. $\rho_S(\%)$ is the proportion of mentions with explicit version declarations. $\rho_V(\%)$ is the proportion of declared versions resolvable to published PyPI releases. $\rho_U(\%)$ is the share of resolved versions with at least one known CVE. $\rho_H(\%)$ reports the fraction of vulnerable versions rated *Critical* or *High*. $\tau_U(\%)$ is the proportion of tasks exposed to at least one vulnerability.

	Model	#Tasks	#LibUses	$\rho_S(\%)$	$\rho_V(\%)$	$\rho_U(\%)$	$\rho_H(\%)$	$\tau_U(\%)$
Explicit	GPT-5.4	1000	2728	6.45	98.86	21.96	310/427(72.60%)	28.10
	Claude-Sonnet-4.6	1000	2991	15.45	95.02	23.91	393/536(73.32%)	33.80
	Gemini-3.1-Pro	1000	2689	8.48	100.00	24.58	375/495(75.76%)	30.60
	DeepSeek-V3.2	1000	3504	51.34	98.00	16.30	63/521(12.09%)	23.20
	Kimi-K2.5	1000	4840	45.91	98.96	26.78	808/1024(78.91%)	35.60
	Qwen3.5-397B	1000	2922	40.49	99.32	26.56	404/575(70.26%)	36.90
	Qwen3-235B	1000	3104	43.49	97.33	26.90	446/610(73.11%)	36.90
	Qwen3-30B	1000	2700	59.19	91.74	32.37	474/686(69.10%)	44.30
	MiniMax-M2.5	1000	2804	19.26	98.89	22.11	321/436(73.62%)	30.60
	Llama-4-Scout	1000	2914	7.69	94.20	17.16	228/350(65.14%)	21.80
Inline	GPT-5.4	1000	2387	95.18	91.29	40.51	565/827(68.32%)	46.20
	Claude-Sonnet-4.6	1000	2836	88.43	85.61	38.58	588/937(62.75%)	50.50
	Gemini-3.1-Pro	1000	2255	91.26	86.15	42.13	568/855(66.43%)	50.70
	DeepSeek-V3.2	1000	2885	70.26	85.64	40.76	699/988(70.75%)	54.00
	Kimi-K2.5	1000	3532	71.89	89.88	45.78	979/1396(70.13%)	55.70
	Qwen3.5-397B	1000	2414	94.16	83.41	40.35	515/803(64.13%)	50.40
	Qwen3-235B	1000	2612	62.29	83.16	37.60	544/803(67.75%)	51.70
	Qwen3-30B	1000	2676	26.83	79.39	32.85	421/565(74.51%)	36.70
	MiniMax-M2.5	1000	2532	91.15	87.52	45.42	730/1028(71.01%)	52.10
	Llama-4-Scout	1000	2866	57.92	81.81	38.59	677/954(70.96%)	51.70

5.1 RQ1: Version Specification Behavior

RQ1 asks whether LLMs annotate dependencies with version identifiers and whether those identifiers correspond to valid published releases. To answer this, we examine four aspects in turn: specification rates across models and modes, the concentration of version choices around particular releases, the validity of specified identifiers, and the temporal lag between specified versions and each model’s knowledge cutoff.

5.1.1 Specification Rate. Prompting mode, rather than model capability, is the dominant driver of version-annotation behavior. As shown in Table 4, under inline mode, seven of the ten models specify versions for more than 70% of all library uses, with the highest-annotating model reaching 95.18%. Qwen3-30B is the sole outlier, annotating only 26.83% of library uses despite belonging to the same model family as peers that annotate more than 60% of their references. The within-family contrast is striking: the two Qwen3 generations from the same vendor differ by nearly 32 percentage points in inline annotation rate, a gap far larger than what parameter count alone can explain. This suggests that generation-level training differences outweigh raw scale in determining annotation frequency.

Under explicit mode, the same models largely abandon version constraints. The three closed-source models annotate fewer than 16% of library uses, with the lowest reaching just 6.45% as shown in Table 4. When generating a `requirements.txt` file, these models routinely produce bare

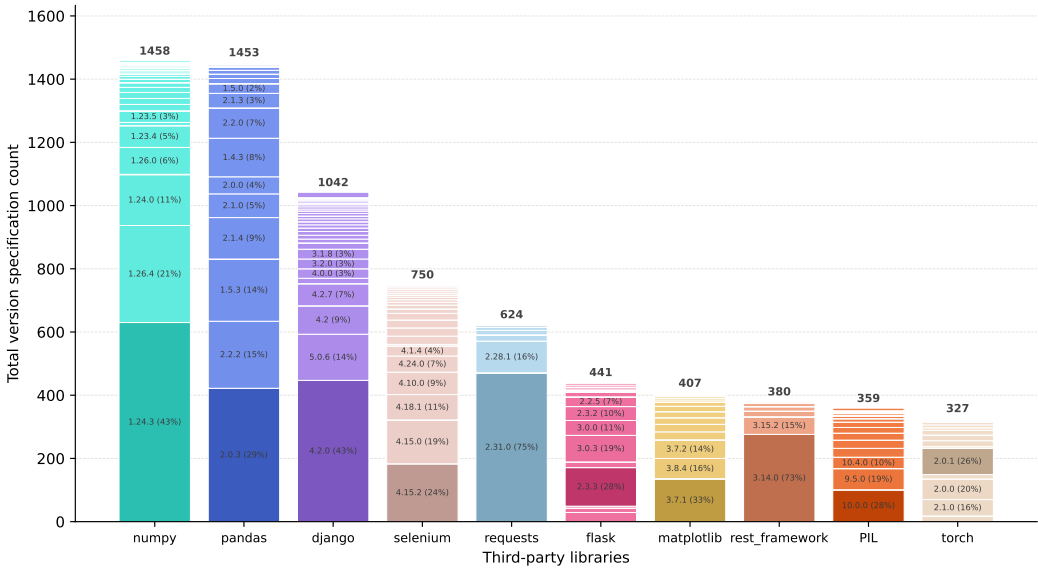


Fig. 3. Specified version distribution for the top-10 most frequently annotated TPLs under inline mode. Each bar shows the total annotation count, and color segments represent individual version shares. Version labels are shown for segments exceeding 2% share.

library names with no version specifier, directly undermining dependency reproducibility. Open-source models show wider variation under explicit mode, ranging from 7.69% to 59.19%. Notably, Qwen3-30B inverts the pattern seen in every other model: it achieves the highest explicit-mode rate while producing the lowest inline rate, indicating that its annotation behavior is sensitive to output format in the opposite direction from its peers. Within the Qwen family, this inversion is progressive: the two larger models each see their annotation rate fall by 20–50 percentage points when switching from inline to explicit mode, while Qwen3-30B shows the opposite, rising by more than 32 points. The smallest model thus behaves more like a structured manifest generator than an inline annotator. This behavioral divergence does not obviously track model capacity.

Based on the results, these within- and across-family patterns point to the same conclusion that format affordance, not model capability, determines whether versions are annotated. The inline-explicit gap reaches as wide as 88.73 percentage points for GPT-5.4. When the output format makes version annotation syntactically natural, most models comply at high rates. When the format permits omitting versions, the same models largely do so. Projects that rely on LLM-generated dependency manifests in explicit mode cannot assume that version information will be present.

5.1.2 Version Concentration. LLMs converge on a small number of versions per library, amplifying any security risk those versions carry. Figure 3 shows the distribution of inline-mode version annotations for the ten most frequently annotated TPLs, aggregated across all models. Across these libraries, version choices are heavily concentrated: 1.24.3 accounts for 43% of all numpy annotations, 43% of LLMs annotate django with 4.2.0, and 75% of requests were labeled as 2.31.0. The django rest framework (rest_framework) distribution is even more extreme, with 3.14.0 accounting for 73% of all annotations for that library. This convergence is not explained by a lack of available releases: requests alone has over 50 published versions on PyPI, yet models overwhelmingly select a single one.

This raises security implications that if a dominant version carries known CVEs, a large fraction of all LLM-generated code referencing that library inherits the same vulnerability, with no per-task variation. The convergence is thus not merely a stylistic observation but a structural risk amplifier.

5.1.3 Validity of Specified Versions. Most inline-specified versions are valid PyPI releases, but a consistent invalidity rate of 7–16% persists across all models. Across 26,691 inline library uses, 2,998 version strings (11.23%) resolve to no published PyPI release. At the model level, Qwen3-30B has the lowest invalidity rate of 7.32% and Qwen3.5-397B the highest of 16.28%. Within the Qwen family, inline validity is remarkably stable despite large differences in specification rate: all three Qwen models cluster within four percentage points of each other in validity, confirming that version-knowledge accuracy is not strongly coupled to annotation frequency. Under explicit mode, validity rates are uniformly higher, at 91.74%–100.00%, but this partly reflects the version-resolution mechanism: library uses without a specifier are resolved to the latest PyPI release, which is valid by construction. Inline validity therefore constitutes the more informative measure of model-level version knowledge.

Inspection of the 2,998 invalid strings reveals two structural categories. *Version hallucination* counts for 91.96% of invalidity at 2,757 cases: a version string that follows the library’s versioning convention but was never released. In such cases, models generate version identifiers as if they were valid releases. A recurring example is `opencv-python==4.8.0`, which appears 16 times each in MiniMax-M2.5 and Qwen3-235B output and 14 times in Qwen3.5-397B output. The underlying error is a suffix omission: OpenCV uses four-part version identifiers, e.g., `4.8.0.74`, `4.8.0.76`, but models consistently generate the truncated form `4.8.0`, which does not exist as a published release. This suffix-omission pattern is consistent across model families, indicating that it reflects a shared training-signal artifact rather than any individual model’s idiosyncrasy. The second category is *cross-library confusion* that has 241 cases (8.04%), where the resolved package name is not found on PyPI because the import name and the package name diverge. Models use the Python import identifier as a PyPI package name. Examples include `scikit_learn` for `scikit-learn`, `azure_storage` for `azure-storage-blob`, and `bio` for `biopython`. Some of these failures are attributable to gaps in the `pipreqs` import-to-package mapping used in Stage (2) of §3.4: packages not covered by the mapping table are resolved by their raw import name, which may differ from their PyPI name.

5.1.4 Version Recency. All ten models specify versions that substantially predate their knowledge cutoffs, with median lags ranging from 9 to 31 months. Figure 4 shows the release-date distribution of inline-mode specified versions per model. Kimi-K2.5 has the shortest lag: its median specified version was released in July 2023, nine months before its April 2024 knowledge cutoff. Several other models fall in the 14–17 month range. For example, GPT-5.4’s median specified version predates its knowledge cutoff by approximately 16 months. At the far end, Qwen3.5-397B has a 31-month lag: its knowledge cutoff is January 2026, yet its median specified version was released in June 2023. Parametric recency does not translate into version recency: despite carrying the most recent training cutoff among all evaluated models, Qwen3.5-397B does not produce the most recent version choices.

The Qwen3 within-family comparison is equally instructive. Qwen3-235B and Qwen3-30B share an identical December 2024 cutoff, yet their median release dates are March 2023 and May 2023, yielding lags of 21 and 19 months respectively. Their distributions are similar despite a 7× difference in active parameter count. The newer-generation Qwen3.5-397B, by contrast, lags its own cutoff by 31 months, substantially more than either Qwen3 model, confirming that the lag is shaped by training data composition rather than model scale or recency.

Several models produce outlier versions released after their own knowledge cutoffs. GPT-5.4 and Qwen3.5-397B are the only exceptions with zero post-cutoff versions. Among the remaining

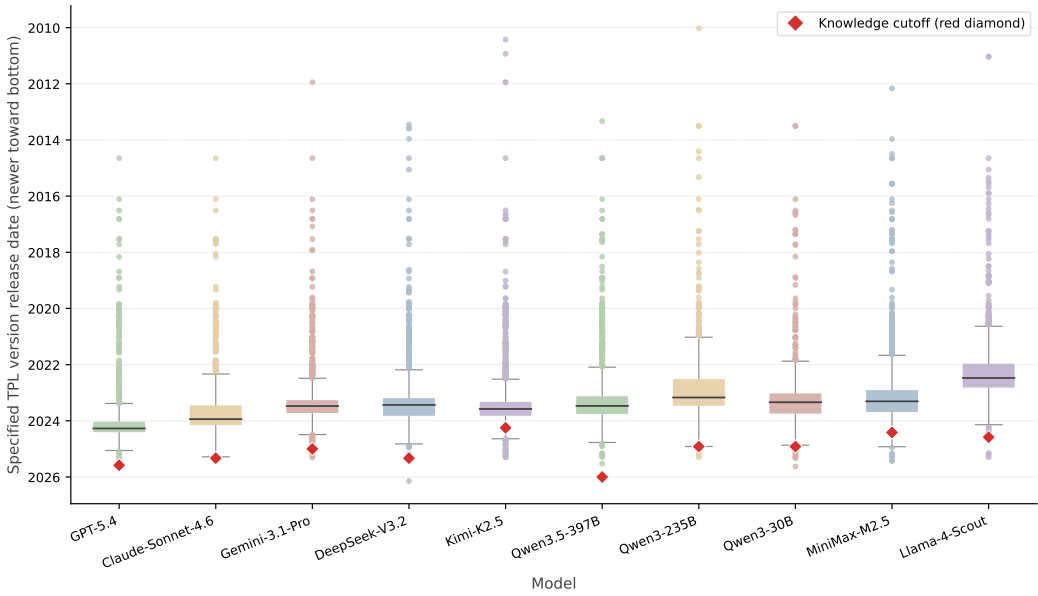


Fig. 4. Distribution of release dates of LLM-specified TPL versions per model under inline mode. The newer dates appear toward the bottom. Red diamonds mark each model’s knowledge cutoff date. For all ten models, the median specified version release date falls above (older than) the cutoff diamond.

eight models, post-cutoff rates range from near zero to 3.75%, with Kimi-K2.5 being the most frequent extrapolator at 95 post-cutoff version strings. The most widespread post-cutoff string is `numpy==2.2.5`, released April 2025, which appears across six models whose cutoffs range from April 2024 to January 2025. These strings cannot have been learned from training data. These models are extrapolating plausible next version numbers for packages they have never seen released. That the extrapolations converge on the same high-profile release suggests models infer near-future version numbers from observed versioning patterns rather than generating them arbitrarily.

In summary, the median lags of 9 to 31 months across all models are consistent with a training-signal hypothesis: a version released years before the cutoff accumulates far more representation in documentation and dependency files than a recently released version, biasing LLM choices toward a popular version window that substantially precedes the release frontier.

Summary: Under inline prompting, LLMs specify version identifiers for 26.83%–95.18% of library references; under explicit prompting, the same models specify only 6.45%–59.19%, with the gap driven by format affordance rather than consistent engineering intent. Among inline-specified versions, 79.39%–91.29% correspond to valid PyPI releases, with the remainder attributable primarily to version hallucination. Annotations concentrate around a small number of dominant versions per library. Those versions systematically lag each model’s knowledge cutoff by 9 to 31 months, reflecting uneven representation of version-specific data in model training.

5.2 RQ2: Version Vulnerability Exposure

Having established that LLMs specify dependency versions at high rates under inline prompting and that most specified versions are valid PyPI releases, we now ask what the security implications of those choices are. RQ2 measures how frequently the specified versions carry known CVE vulnerabilities and characterizes the severity, structural origins, and temporal properties of that

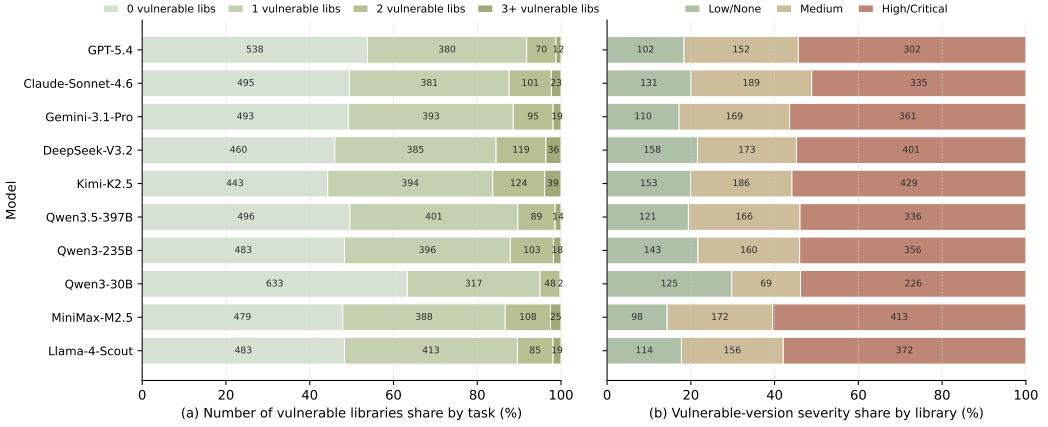


Fig. 5. Per-model vulnerability exposure profile under inline mode. (a): Distribution of the number of vulnerable library versions per task. (b): Severity composition (*None/Low/Medium/High/Critical*) of all vulnerable versions specified by each model.

exposure. We focus on inline mode throughout this section. As §5.1 showed, explicit mode produces so few version annotations that model-level vulnerability comparisons are less informative.

5.2.1 Overall Vulnerability Exposure. LLMs expose 36.70%–55.70% of tasks to known vulnerabilities through their version choices. As shown in Table 4, task-level vulnerability exposure under inline prompting ranges from 36.70% for Qwen3-30B to 55.70% for Kimi-K2.5. Figure 5(a) shows that multi-library vulnerability is common among exposed tasks. For Kimi-K2.5, the model with the highest exposure, nearly 30% of its exposed tasks contain two or more vulnerable library versions simultaneously. For Qwen3-30B, the model with the lowest exposure is around 14%. Vulnerability exposure is therefore not limited to isolated single-library incidents but frequently involves concurrent vulnerabilities across multiple dependencies in the same generated snippet.

High-severity vulnerabilities dominate across all models. As shown in Figure 5(b) and Table 4, *Critical* and *High* severity CVEs account for 62.75%–74.51% of all vulnerable versions. For context, the OSV dataset baseline distributes PyPI vulnerability severity as 47.86% Low/None, 23.24% Medium, and 28.90% High/Critical across all known records, without filtering by LLM preference. LLM-recommended versions therefore carry a disproportionate share of high-severity CVEs relative to the overall vulnerability landscape. This overrepresentation suggests that the versions LLMs prefer are not merely old, but are specifically those that accumulated the most severe unpatched vulnerabilities before fixes became widely adopted.

5.2.2 Vulnerable Version Convergence. Vulnerable version choices are shared across model families, not idiosyncratic to individual models. Figure 6 plots the top-10 most frequently specified vulnerable (ℓ, v) pairs as a heatmap across all ten models. Across 6,378 total vulnerable version assignments spanning 1,289 unique vulnerable (ℓ, v) pairs, the top-10 entries alone account for 26.34% of all assignments. The top five entries are each specified by at least nine of the ten models, while seven of the ten top entries meet this threshold. The single most convergent pair is `django==6.0.1`, specified 400 times across all ten models.

Uniform row coloring is visible for the top entries, confirming that convergence rather than model-specific preference drives the most common vulnerable choices. The lower entries show more model-specific concentration, but the five most convergent pairs, `requests==2.31.0`, `django==6.0.1`, `flask==3.1.2`, `flask==2.3.3`, and `djangorestframework==3.14.0`, each appear in at least nine

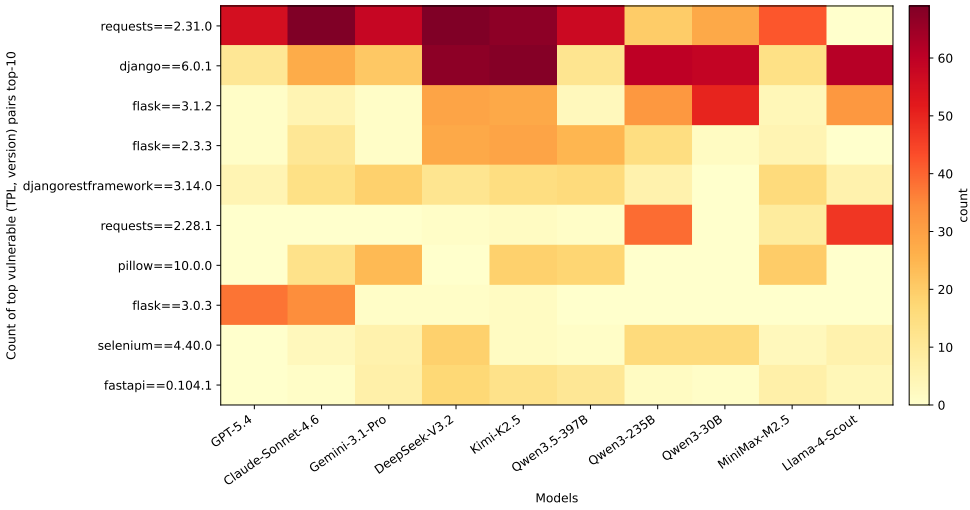


Fig. 6. Cross-model convergence on the most frequently specified vulnerable (ℓ, v) pairs under inline mode. Cell color encodes specification count; rows are sorted by total count across all models. Uniform row coloring indicates cross-model consensus on the same vulnerable version. Concentrated single-column coloring indicates model-specific preference.

of the ten models' outputs, spanning both closed-source and open-source families. This breadth rules out any single training pipeline as the source of the bias. The pattern directly corroborates the version-concentration finding in §5.1.2: the same narrow set of popular releases that LLMs overwhelmingly select also happens to carry the most prevalent CVEs, confirming that high version concentration translates directly into concentrated vulnerability exposure.

5.2.3 Temporal Awareness of Known Vulnerabilities. The convergence result raises a further question: were the CVEs carried by LLM-specified versions publicly disclosed before training, making the exposure in principle avoidable, or only disclosed after?

Most CVEs carried by LLM-specified versions were publicly disclosed before each model's training cutoff. Figure 7 plots Δ = disclosure – cutoff in months. Negative values indicate pre-cutoff disclosure. As shown in Figure 7, across all 1,410 unique CVE identifiers observed in the study, 72.27%–91.37% of the unique CVEs associated with each model's specified versions were disclosed before that model's knowledge cutoff. The proportion is highest for Qwen3.5-397B at 91.37%, which carries the most recent knowledge cutoff of January 2026, and lowest for Gemini-3.1-Pro at 72.27%, which has an earlier cutoff of January 2025. Aggregated across all models, 81.8% of unique CVE-model pairs correspond to vulnerabilities that were part of the public record at training time.

For all ten models, the bulk of mass in Figure 7 lies left of $\Delta = 0$, with long left tails extending to $\Delta \approx -150$ months. Several outlier CVEs dating to 2007–2008 are visible at the extreme left. These are decade-old vulnerabilities still carried by widely-used library versions, present in LLM-recommended code because those versions accumulated extraordinary web presence long before the vulnerabilities were well-known. LLM-introduced vulnerability exposure is therefore not primarily a consequence of training-data latency. The relevant CVEs were accessible during training, making the exposure an avoidable rather than an inherent limitation.

5.2.4 Availability of Safe Alternatives. Version-level vulnerability exposure cannot be corrected by minor version substitution. The findings above establish that LLM-specified versions

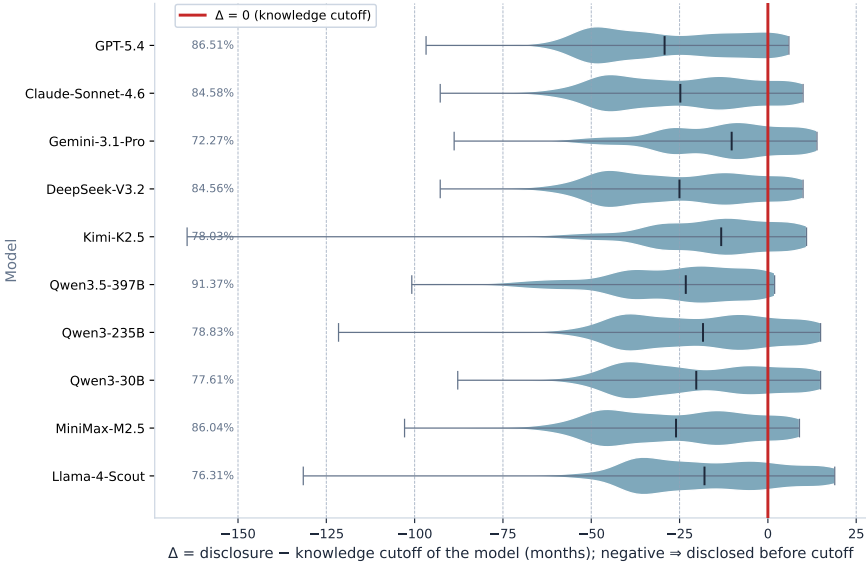


Fig. 7. Distribution of the lag between CVE disclosure dates and each model's knowledge cutoff (Δ = disclosure - cutoff, in months) for all vulnerabilities carried by LLM-specified versions under inline mode. Negative values indicate that the CVE was publicly disclosed before the model's training cutoff. The red line marks $\Delta = 0$. The bulk of each distribution lies left of zero, showing that the vast majority of vulnerabilities in LLM-recommended versions were already part of the public record at training time.

frequently carry known CVEs. A natural question is whether exposure could be resolved by choosing a patch version of the same library. We examine whether, for each LLM-specified vulnerable (ℓ, v) pair, a CVE-free alternative exists within the same major version series. Such a substitution would be minimal, preserving API compatibility in most cases. Out of 1,289 unique vulnerable (ℓ, v) pairs identified across all models, only 63 have at least one CVE-free release in the same major version branch. For 95.11% of vulnerable pairs, the entire same-major version branch is affected by at least one CVE, leaving no safe same-major alternative available. Developers who discover a vulnerable LLM-specified version cannot simply increment the patch number. They face a cross-major migration or a deliberate security-aware version selection process. This structural characteristic makes the problem qualitatively different from ordinary version drift and underscores the need for external tooling rather than local version adjustment.

Summary: Under inline prompting, 36.70%–55.70% of tasks contain at least one LLM-specified library version carrying a known CVE, with 62.75%–74.51% of vulnerable versions rated *Critical* or *High* severity. The most frequently specified vulnerable versions are shared across all models, indicating a systematic bias rather than model-specific error. The associated CVEs were publicly disclosed before each model's knowledge cutoff in 72%–91% of cases, making the exposure avoidable in principle. Finally, 95.11% of vulnerable (ℓ, v) pairs have no CVE-free alternative within the same major version branch, showing a structural problem.

5.3 RQ3: Version-Code Compatibility

RQ1 and RQ2 established that LLMs frequently pin specific library versions and that those versions often carry known CVEs. RQ3 asks a complementary question. Even setting security aside, are the pinned versions actually compatible with the generated code? We assess this through two stages:

Table 5. Static compatibility rates and quadrant task breakdown. $\tau_C(\%)$ is the passing rate for static type check. Quadrant entries partition tasks by the intersection of compatibility and vulnerability status.

	Model	$\tau_C(\%)$	Safe $\cap$ Compat	Unsafe $\cap$ Compat	Safe $\cap$ Incompat	Unsafe $\cap$ Incompat
Explicit	GPT-5.4	63.20	49.30	13.90	22.60	14.20
	Claude-Sonnet-4.6	53.40	40.50	12.90	25.70	20.90
	Gemini-3.1-Pro	66.70	49.60	17.10	19.80	13.50
	DeepSeek-V3.2	48.50	40.60	7.90	36.20	15.30
	Kimi-K2.5	51.90	37.90	14.00	26.50	21.60
	Qwen3.5-397B	62.30	42.60	19.70	20.50	17.20
	Qwen3-235B	52.20	38.20	14.00	24.90	22.90
	Qwen3-30B	45.90	31.20	14.70	24.50	29.60
	MiniMax-M2.5	55.20	42.00	13.20	27.40	17.40
	Llama-4-Scout	45.30	37.60	7.70	40.60	14.10
	GPT-5.4	63.20	36.90	26.30	16.90	19.90
	Claude-Sonnet-4.6	46.00	27.20	18.80	22.30	31.70
	Gemini-3.1-Pro	50.60	26.50	24.10	22.80	26.60
Inline	DeepSeek-V3.2	37.60	19.80	17.80	26.20	36.20
	Kimi-K2.5	41.80	20.90	20.90	23.40	34.80
	Qwen3.5-397B	46.10	23.00	23.10	26.60	27.30
	Qwen3-235B	34.40	19.30	15.10	29.00	36.60
	Qwen3-30B	44.60	32.10	12.50	31.20	24.20
	MiniMax-M2.5	19.70	12.10	7.60	35.80	44.50
	Llama-4-Scout	25.40	15.20	10.20	33.10	41.50

static compatibility, assessed via installation verification and ty static type checking, and dynamic compatibility, assessed via BigCodeBench test suite execution.

5.3.1 Static Compatibility. Inline prompting produces substantially lower static compatibility rates than explicit prompting. This difference reflects the distinct version-resolution mechanisms of the two modes. Table 5 reports per-model compatibility rates. Under explicit prompting, rates range from 45.30% to 66.70%. Because most models annotate fewer than 20% of library references in explicit mode, the majority of dependencies are resolved to their latest PyPI release at the study’s temporal anchor. Latest releases install successfully by construction, inflating the apparent compatibility rate. Under inline prompting, where models pin specific versions on every import, compatibility rates fall considerably, ranging from 19.70% to 63.20% across models. Most inline models fall below 50%. GPT-5.4 is the only model that achieves equal compatibility across both modes, at 63.20%.

The **Unsafe $\cap$ Incompat** quadrant captures tasks that simultaneously expose the user to a known CVE and fail static checking. Under inline prompting, this quadrant accounts for 19.90% to 44.50% of tasks across models. Only GPT-5.4 achieves a **Safe $\cap$ Compatible** rate above 30%, at 36.90%. All other inline models fall below 27%. Fully safe and executable code is therefore the exception rather than the norm under inline version pinning.

5.3.2 Compatibility Error Analysis. Installation failure is the dominant cause of static incompatibility under inline prompting. Table 6 reports the primary diagnostic for each incompatible task. Every run in each task of PinTrace is processed in a new isolated temporary Python venv, we install the LLM specified TPLs at their versions and run the corresponding checks as illustrated in §3.4. Under inline mode, dependency installation error is the single largest category for every model, ranging from 124 tasks for GPT-5.4 to 472 for Qwen3-235B, showing LLMs specify versions that are not even installable. API-level diagnostics reported after a successful installation,

Table 6. Static compatibility check failure diagnostics. For each incompatible task, the first reported ty check rule is used as its primary diagnostic label. Columns show the six globally most frequent error categories plus a catch-all. **InstErr**: dependency installation failure, **UnrAttr**: unresolved attribute reference, **UnrImp**: unresolved import, **InvSyn**: invalid syntax, **InvArg**: invalid argument type, **UnrRef**: unresolved reference.

	Model	InstErr	UnrAttr	UnrImp	InvSyn	InvArg	UnrRef	Others	Total
Explicit	GPT-5.4	129	56	37	25	26	7	88	368
	Claude-Sonnet-4.6	221	70	57	8	23	9	78	466
	Gemini-3.1-Pro	118	70	40	5	19	1	80	333
	DeepSeek-V3.2	309	51	40	9	24	9	73	515
	Kimi-K2.5	219	60	43	34	34	17	74	481
	Qwen3.5-397B	174	52	49	4	22	3	73	377
	Qwen3-235B	229	58	57	11	30	15	78	478
	Qwen3-30B	327	51	58	10	15	28	52	541
	MiniMax-M2.5	136	60	70	50	21	28	83	448
	Llama-4-Scout	204	73	109	10	29	23	99	547
Inline	GPT-5.4	124	58	39	2	55	11	79	368
	Claude-Sonnet-4.6	241	44	40	126	20	14	55	540
	Gemini-3.1-Pro	343	51	31	0	14	2	53	494
	DeepSeek-V3.2	462	39	46	11	18	8	40	624
	Kimi-K2.5	406	45	42	7	18	10	54	582
	Qwen3.5-397B	388	37	54	3	11	5	41	539
	Qwen3-235B	472	50	57	7	14	12	44	656
	Qwen3-30B	253	57	98	14	21	41	70	554
	MiniMax-M2.5	400	17	18	337	5	1	25	803
	Llama-4-Scout	452	70	73	35	14	54	48	746

namely unresolved-attribute, unresolved-import, and invalid-argument-type, reported as UnrAttr, UnrImp, and InvArg in Table 6, together account for a smaller but consistent share. These reflect cases where the dependency set installs, but the generated code references API surfaces that do not exist in the pinned version. This shows LLMs cannot fully correspond to the API usage of TPLs with their version.

Among installation failures, dependency resolution conflict is the most prevalent sub-cause across all models, accounting for 84–249 cases per model under inline prompting. These conflicts arise when pinned versions impose mutually incompatible transitive dependency constraints that the resolver cannot satisfy simultaneously. A distinct cluster, `missing_distutils`, accounts for 8–134 inline failures per model. It reflects a systematic version-era mismatch: packages whose metadata was written before Python removed `distutils` from the standard library in version 3.12 fail to install in the evaluation environment. This pattern is a direct consequence of the temporal lag documented in §5.1.4: LLMs preferentially specify older versions, and those versions disproportionately carry dependencies on a module that no longer exists. Under explicit prompting, `missing_distutils` failures are far rarer, as latest-version fallback resolution tends to select releases that have already dropped the deprecated dependency.

Two models exhibit anomalously high invalid-syntax counts under inline mode. MiniMax-M2.5 accounts for 337 such tasks and Claude-Sonnet-4.6 for 126. This pattern is distinct from the version-level failures that dominate elsewhere. It indicates that these models occasionally generate syntactically malformed code when producing inline version annotations, introducing a code-quality failure independent of any version choice. Taken together, the error taxonomy shows

Table 7. Measurement results of BigCodeBench test case execution. #Pass, #Fail, and #Error report task-level test outcomes, #Total denotes the number of measured tasks, and PassRate(%) is the pass rate among them.

	Model	#Pass	#Fail	#Error	#Total	PassRate(%)
Explicit	GPT-5.4	347	372	5	724	47.93
	Claude-Sonnet-4.6	353	346	25	724	48.76
	Gemini-3.1-Pro	372	342	10	724	51.38
	DeepSeek-V3.2	221	265	238	724	30.52
	Kimi-K2.5	307	396	21	724	42.40
	Qwen3.5-397B	358	362	4	724	49.45
	Qwen3-235B	271	299	154	724	37.43
	Qwen3-30B	220	323	181	724	30.39
	MiniMax-M2.5	316	390	18	724	43.65
	Llama-4-Scout	234	392	98	724	32.32
Inline	GPT-5.4	352	351	21	724	48.62
	Claude-Sonnet-4.6	53	60	611	724	7.32
	Gemini-3.1-Pro	132	146	446	724	18.23
	DeepSeek-V3.2	113	129	482	724	15.61
	Kimi-K2.5	98	148	478	724	13.54
	Qwen3.5-397B	71	79	574	724	9.81
	Qwen3-235B	50	68	606	724	6.91
	Qwen3-30B	117	183	424	724	16.16
	MiniMax-M2.5	47	73	604	724	6.49
	Llama-4-Scout	59	114	551	724	8.15

that version selection, not code logic, accounts for the overwhelming share of incompatibility failures under inline mode.

5.3.3 Dynamic Compatibility. **Dynamic pass rates confirm and sharpen the static picture: inline mode fails predominantly at the installation stage rather than at the logic stage.**

Table 7 reports BigCodeBench execution outcomes. Under explicit prompting, pass rates range from 30.39% to 51.38%, broadly consistent with the static compatibility rates in §5.3.1. Under inline prompting, pass rates collapse to 6.49%–48.62%, with nine of ten models below 20%. The critical diagnostic is the #Error column, which captures tasks that terminate before any test assertion can run, typically due to installation failure. Under inline mode, error counts reach 424–611 per model across the 724-task benchmark, meaning that the majority of tasks never execute at all. Under explicit prompting, error counts are overwhelmingly lower, reaching single digits for most closed-source models, which confirms that the inline-explicit gap is attributable to version-level incompatibility rather than any difference in code generation quality between the two modes.

GPT-5.4 is the notable exception: its inline pass rate of 48.62% nearly matches its explicit rate of 47.93%, with only 21 error-category tasks. This is consistent with its static compatibility advantage observed in §5.3.1, where it was the only model achieving above 60% in both modes.

5.3.4 Relationship Between Static and Dynamic Compatibility. **Static compatibility is a strong positive predictor of dynamic success, but the two measures are not interchangeable.**

Figure 8 illustrates the relationship for GPT-5.4 under inline mode, the model with the highest inline pass rate and therefore the most interpretable signal. Of all 724 tasks, 77.35% are classified as static compatible and 22.65% as incompatible. Among the compatible tasks, 40.47% flow through to a runtime pass, while 36.88% fail at runtime despite passing static checking. Within the compatible

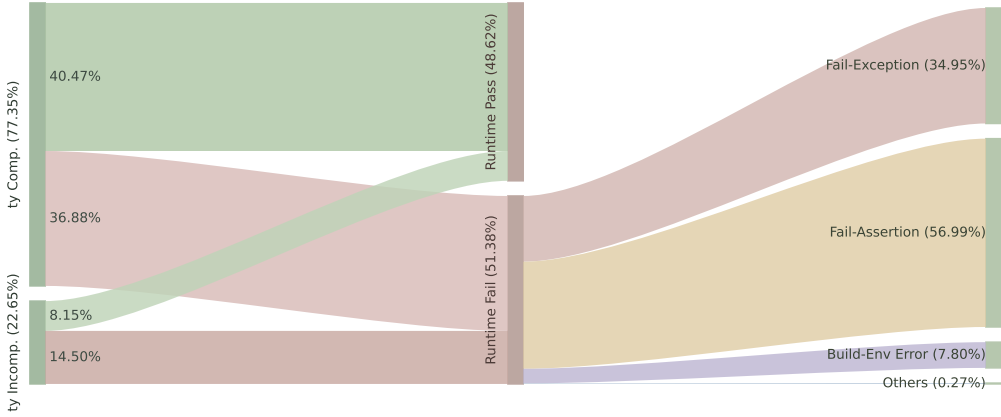


Fig. 8. Flow from static compatibility outcomes to dynamic execution results across BigCodeBench tasks under inline mode, shown for GPT-5.4. Left nodes represent ty static verdict, right nodes represent BCB runtime outcome. Proportions are relative to all 724 tasks in the first and second columns. The proportions in the last column are relative to the 372 runtime fail tasks.

set, runtime failures decompose into four categories. Fail-Assertion accounts for the largest share at 56.99%, representing tasks where the API surface is correct but the generated code is functionally wrong. Fail-Exception covers uncaught runtime exceptions at 34.95%. Build/Env Error captures environment setup failures that occur after static checking passes, at 7.80%. The remaining 0.27% falls into other categories. All of these errors that static type checking cannot detect in advance.

In the reverse direction, 8.15% of tasks classified as ty static incompatible nonetheless pass at runtime, indicating that a fraction of static type errors are false positives or reflect API differences that do not affect actual execution paths. Static compatibility therefore substantially overestimates the runtime pass rate when the incompatible fraction is large, as is the case for most models under inline mode. It does, however, provide a reliable lower bound on the fraction of tasks that will fail.

Summary: Static compatibility rates range from 19.70% to 63.20% under inline prompting and 45.30%–66.70% under explicit prompting, with dependency installation failure due to temporal lag identified in RQ1 as the dominant cause. Specifically, LLMs preferentially recommend older versions that disproportionately fail to install in modern Python environments. Dynamic verification confirms the pattern, with inline pass rates collapsing to 6.49%–48.62% as version incompatibilities prevent most tasks from executing at all.

6 Robustness, Diagnosis, and Mitigation

The main findings in §5 raise three follow-up questions: Do the compatibility results hold across different Python runtime environments? Are the installation failures caused by version selection or by the generated code itself? Can prompt-level interventions reduce the identified risks? §6.1 addresses the first question by repeating the full compatibility pipeline across four Python versions. §6.2 addresses the second through a controlled substitution experiment that holds generated code fixed and varies only the specified versions. §6.3 addresses the third by evaluating four progressively stronger prompt-level interventions.

Table 8. Static compatibility (τ_C , %) and BigCodeBench dynamic pass rate (%) on the BigCodeBench dataset across Python runtime versions.

	Model	3.8		3.10		3.12		3.14	
		τ_C	BCB	τ_C	BCB	τ_C	BCB	τ_C	BCB
Explicit	GPT-5.4	2.95	1.72	14.76	8.12	91.64	49.32	91.14	49.08
	Claude-Sonnet-4.6	8.00	5.04	16.24	8.49	88.81	49.32	88.56	49.20
	Gemini-3.1-Pro	3.94	2.21	15.99	8.12	92.74	52.03	92.25	51.54
	DeepSeek-V3.2	44.16	23.49	56.58	27.43	63.84	31.86	54.98	28.17
	Kimi-K2.5	81.67	39.85	74.54	40.71	88.56	43.42	87.33	42.80
	Qwen3.5-397B	4.67	2.83	15.74	8.86	92.74	50.55	92.50	50.31
	Qwen3-235B	44.90	22.02	45.51	23.25	74.42	38.50	74.54	38.50
	Qwen3-30B	51.91	27.18	52.15	27.80	66.54	31.73	65.44	31.12
	MiniMax-M2.5	10.46	5.41	19.31	9.84	91.14	44.53	91.14	44.28
	Llama-4-Scout	6.40	3.57	16.61	7.26	80.81	33.21	80.69	32.96
Inline	GPT-5.4	9.47	4.92	74.78	46.86	78.72	49.69	27.80	12.55
	Claude-Sonnet-4.6	75.28	42.44	87.95	30.14	21.89	9.59	17.34	9.10
	Gemini-3.1-Pro	63.35	35.30	92.25	35.67	41.08	21.03	19.19	8.36
	DeepSeek-V3.2	57.56	28.04	90.77	38.25	34.19	16.73	13.90	5.66
	Kimi-K2.5	59.78	31.12	90.65	32.47	37.27	14.15	15.62	6.52
	Qwen3.5-397B	81.43	45.26	93.60	28.78	25.95	10.95	18.20	8.49
	Qwen3-235B	62.73	29.52	66.79	17.59	32.84	8.24	22.51	9.72
	Qwen3-30B	48.59	24.11	62.12	22.14	45.51	17.84	36.04	15.74
	MiniMax-M2.5	79.83	40.47	91.88	30.75	21.28	8.12	15.74	6.77
	Llama-4-Scout	61.99	25.95	68.02	19.19	26.20	9.10	25.95	9.23

6.1 Robustness Across Python Environments

The main experiments use Python 3.12 as the execution environment. To assess whether the compatibility findings generalize across runtime versions, we repeat the full Stage (5) pipeline under Python 3.8, 3.10, and 3.14 on the BigCodeBench dataset. This dataset supports both static and dynamic evaluation on the same task set. Vulnerability exposure is not re-evaluated. It is a static property of (ℓ, v) pairs and is invariant to the runtime environment.

Table 8 reports static compatibility and dynamic pass rates across all four Python versions and both prompting modes.

The two prompting modes respond to Python version in opposite directions, revealing that the compatibility pattern is driven by LLM version choices rather than by the runtime itself. Under inline mode, both static compatibility and dynamic pass rates peak around Python 3.10 and decline sharply at 3.12 and beyond. LLMs preferentially specify library versions released before 2022, and many of those versions carry build-time dependencies on `distutils`, which was available through Python 3.10 but removed from the standard library in Python 3.12. For DeepSeek-V3.2, inline τ_C falls from 90.77% at Python 3.10 to 34.19% at 3.12. The BCB pass rate drops from 38.25% to 16.73% over the same transition. Under explicit mode, both metrics increase monotonically from Python 3.8 to 3.12 and then stabilize, because LLM outputs of most explicit mode tasks rely on the latest version fallback, and newer releases have progressively dropped support for older versions of Python. The reversal between the two mode trends confirms that the sensitivity to Python version is entirely a consequence of version choices made by the LLM, not a property of the runtime.

Static and dynamic results track each other closely across all Python versions. For every model and every Python version, the BCB dynamic pass rate moves in the same direction as τ_C .

This confirms that installation failure is the governing mechanism: a version that cannot install prevents execution before any test can run.

Cross-model rankings are preserved across Python environments. The ordering of models by compatibility under inline mode is consistent across all four Python versions, matching the ordering reported in §5.3. The choice of Python 3.12 as the primary evaluation environment does not introduce systematic bias in cross-model comparisons.

6.2 Diagnosing Compatibility Failures

Section 5.3 showed that installation failure is the dominant cause of static incompatibility under inline prompting. The remaining question is whether that failure stems from version selection or from the generated code itself imposing unsatisfiable constraints. The two causes require different remedies, so a controlled attribution is necessary.

Experimental design. To isolate version selection as the cause of installation failures, we hold the generated code fixed and test whether swapping the specified version to a neighboring release resolves the failure. We select qualifying tasks from the inline-mode PinTrace results under two conditions: the task must have failed Stage (5) by dependency installation error, and all LLM-specified versions in the task must be CVE-free. The first condition ensures the failure is unambiguously at the dependency resolution layer. The second eliminates security as a confounding variable: if a vulnerable version were included, a neighboring version might resolve the failure by switching to a safer release, entangling compatibility and security attribution.

For each qualifying task, we hold the generated code $G(t, m, p)$ fixed and search for a substitute version. Let $\mathcal{V}_{\text{fail}}(t, m, p) \subseteq \mathcal{V}(t, m, p)$ denote the subset of LLM-specified, version-annotated library with install error. For each $(\ell, v) \in \mathcal{V}_{\text{fail}}$, we enumerate a neighbor version as:

$$\mathcal{N}(\ell, v) = \{ v' \in \Pi(\ell) : |r(v') - r(v)| \leq N \} \quad (4)$$

where $\Pi(\ell)$ is the ordered list of published PyPI releases for library ℓ and $r(\cdot)$ maps a version to its release-sequence index. We set $N = 3$, yielding up to six neighbors per library, a window narrow enough to avoid crossing major API boundaries and introducing breaking changes between versions. Candidate substitutions are formed by replacing the failing version in $\mathcal{V}_{\text{fail}}$ with a neighbor while holding all other versions fixed. To bound the search, we cap the number of evaluated combinations per task at $k = 40$. For each evaluated substitution, we re-run Stage (5). A task is *installable-recovered* if the substitution removes all installation error occurrences, and *compatible-recovered* if it additionally passes ty static type checking. All reported rates are lower bounds, since the neighborhood is finite and may not cover all compatible releases.

Version substitution resolves installation failures in 17.19%–55.00% of qualifying tasks, establishing version selection as the primary cause. Table 9 reports results for all ten models. The compatible rate, the more demanding criterion requiring both installation success and static compatibility, ranges from 7.81% to 46.04%. Recovery rates vary across models, reflecting differences in how badly each model’s version choices deviate from installable releases. The phenomenon is consistent, however: in every case, a neighboring version resolves failures that the originally specified version cannot. Since the generated code is held fixed throughout, any resolution is attributable solely to the version change.

Neighboring-version substitution introduces negligible additional security risk. Across all 64–205 qualifying tasks per model, only five total recovered tasks across all models introduce at least one new CVE in the substitute version. Six models report zero such cases, while four models report one or two each. Compatibility recovery and security safety are therefore largely compatible objectives: the version neighborhood that resolves installation failures is not, in general, a more vulnerable region of the release timeline.

Table 9. Neighbor-version search on PinTrace tasks that have package installation errors. **Qualifying** is the number of tasks meeting the selection criteria. **Installable** counts tasks with a neighbor that removes the installation error (percentage of Qualifying). **Compatible** counts tasks with a neighbor that additionally passes static type-checking (percentage of Qualifying). **Vuln@Reco** counts installable recoveries that introduce at least one new CVE. $\bar{N}_{\text{test}}$ is the mean number of neighbor candidates evaluated per qualifying task. All rates are lower bounds under the configured search budget.

Model	Qualifying	Installable	Compatible	Vuln@Reco	$\bar{N}_{\text{test}}$
GPT-5.4	64	11 (17.19%)	5 (7.81%)	1	14.80
Claude-Sonnet-4.6	92	28 (30.43%)	15 (16.30%)	0	15.12
Gemini-3.1-Pro	177	91 (51.41%)	76 (42.94%)	0	14.58
DeepSeek-V3.2	202	95 (47.03%)	78 (38.61%)	0	16.24
Kimi-K2.5	173	78 (45.09%)	60 (34.68%)	0	18.10
Qwen3.5-397B	202	109 (53.96%)	93 (46.04%)	0	15.70
Qwen3-235B	205	104 (50.73%)	80 (39.02%)	0	18.43
Qwen3-30B	140	77 (55.00%)	57 (40.71%)	1	15.16
MiniMax-M2.5	197	103 (52.28%)	28 (14.21%)	2	20.78
Llama-4-Scout	205	50 (24.39%)	34 (16.59%)	1	21.20

When installation succeeds, static type checking also passes in the majority of cases.

Among tasks where a neighboring version removes the installation error, the fraction that additionally passes the static type check is consistently high. For example, Gemini-3.1-Pro recovers 91 tasks at the installable level, of which 76 (83.5%) also pass static type checking. DeepSeek-V3.2 recovers 95, of which 78 (82.1%) pass. The high conditional type-check rate confirms that the generated code is largely API-consistent once the version constraint is resolved. Thus, the original failure was caused by the version choice, not by a defect in the generated logic.

6.3 Mitigation Probe

Section 6.2 established that version selection is the primary cause of compatibility failures, and §5.2 showed that LLM-specified versions frequently carry known CVEs. We evaluate whether these problems can be reduced through prompt-level interventions, without modifying model weights.

Experimental conditions. **Baseline** (inline) is the standard inline prompt from §5. **Safety instruction** (abl-instruct) appends a natural-language instruction to avoid versions with known CVEs. No structured version information is provided. **Version anchor** (abl-version) injects a pre-computed safe version for each TPL determined by the OSV index and PyPI metadata, following a three-tier priority: the most recent CVE-free release, the release with the fewest known CVEs, or the most recent known release as a last resort. Specifically, 91.33% of TPLs in our dataset were resolved at the safe tier, 8.67% at min-vuln. **Version anchor with RAG** (abl-rag) extends abl-version by additionally retrieving the top-20 most relevant API signatures for each resolved (ℓ, v) pair. API signatures for each resolved (ℓ, v) pair are first extracted from the installed package using griffe [64], a static analysis tool that parses Python source and stubs to produce structured API documentation. The top-20 most relevant signatures are then selected by BM25 retrieval against the task question text and prepended to the prompt.

Safety instruction provides negligible security improvement. As shown in Table 10, across all ten models, abl-instruct reduces ρ_U by at most 2.2 percentage points and τ_U by at most 2.2 percentage points relative to baseline. For three models whose baseline exposure is already partially grounded in version awareness, Gemini-3.1-Pro, DeepSeek-V3.2, and Qwen3.5-397B, ρ_U and τ_U are essentially unchanged or slightly higher under abl-instruct. The negligible improvement is

Table 10. Security metrics across ablation conditions on the PinTrace dataset. $\rho_U(\%)$: library vulnerability rate. $\tau_U(\%)$: task vulnerability exposure.

Model	Baseline		abl-instruct		abl-version		abl-rag	
	ρ_U	τ_U	ρ_U	τ_U	ρ_U	τ_U	ρ_U	τ_U
GPT-5.4	40.51	46.20	38.39	44.00	9.98	10.90	10.47	11.60
Claude-Sonnet-4.6	38.58	50.50	46.55	54.80	11.99	15.00	14.88	16.00
Gemini-3.1-Pro	42.13	50.70	42.76	51.90	12.83	15.80	11.55	13.80
DeepSeek-V3.2	40.76	54.00	40.90	54.60	19.34	25.00	17.91	21.30
Kimi-K2.5	45.78	55.70	44.61	54.90	20.83	22.30	20.06	21.80
Qwen3.5-397B	40.35	50.40	40.39	49.40	12.13	15.80	13.30	16.40
Qwen3-235B	37.60	51.70	36.40	50.80	15.62	17.00	16.55	19.30
Qwen3-30B	32.85	36.70	32.23	36.80	22.17	23.00	24.00	24.20
MiniMax-M2.5	45.42	52.10	45.05	50.20	15.02	19.10	14.87	17.70
Llama-4-Scout	38.59	51.70	38.36	49.90	21.08	27.60	20.53	26.60

consistent with the temporal analysis in §5.2.3: the relevant CVEs were publicly disclosed before each model’s training cutoff, so the model is not unaware of the vulnerability landscape in principle. Rather, it lacks the ability to map version identifiers to CVE records at inference time, and a natural-language instruction cannot compensate for this grounding gap.

Version anchoring substantially reduces but does not eliminate vulnerability exposure.

Under abl-version, library vulnerability rate ρ_U drops to 9.98%–22.17% and τ_U to 10.90%–27.60% across models, as shown in Table 10. These are large reductions relative to baseline, but residual exposure remains. The residual arises when no CVE-free release exists for a given library: the version anchor falls back to the release with the fewest known CVEs rather than a CVE-free one. For such libraries, any pinned version carries at least one CVE, making zero exposure structurally unachievable without excluding the library entirely. However, all ten models report ρ_U values under abl-version that exceed this 8.67% floor of the PinTrace dataset itself, indicating that the models introduce additional TPLs beyond those specified in the injected version list, and some of those self-introduced libraries are themselves pinned to vulnerable versions. abl-rag yields similar security metrics, confirming that the residual vulnerability is a property of the library coverage rather than a consequence of the API-grounding intervention.

Version anchoring systematically improves compatibility. The compatibility effect of the ablation abl-version is far larger than its security effect, as Table 11 shows. On BigCodeBench, τ_C increases from a baseline range of 21.89%–78.72% to 80.92%–93.64% under abl-version, and dynamic pass rates rise from 7.32%–49.69% to 36.31%–54.56%. The improvement is uniform across model families, indicating that the baseline compatibility failures are attributable to version selection rather than to model-level code generation quality.

RAG-augmented prompting provides marginal additional benefit. abl-rag improves τ_C by 0.10–2.05 percentage points over abl-version and dynamic pass rates by at most 1.03 percentage points, as the Δ columns in Table 11 show. For some models, the dynamic pass rate under abl-rag is slightly lower than under abl-version, suggesting that injected API context occasionally misleads rather than assists. Two structural factors limit the approach. First, BM25 retrieval uses the task question text as the query, which describes the programming goal rather than the specific functions the model will invoke; retrieved signatures may therefore cover API surface adjacent to, but not identical to, what the generated code actually needs. Second, injected signatures for one version may conflict with the model’s parametric knowledge of a different version’s API, introducing inconsistency that the model cannot resolve at inference time. The marginal and occasionally

Table 11. Static compatibility (τ_C , %) and BigCodeBench dynamic pass rate (%) across ablation conditions. Values in parentheses give percentage-point change versus Baseline.

Model	Baseline		abl-instruct		abl-version		abl-rag	
	τ_C	BCB	τ_C	BCB	τ_C	BCB	τ_C	BCB
GPT-5.4	77.35	48.62	77.35(+0.00)	46.82(−1.80)	91.27(+13.92)	46.67(−1.95)	91.75(+14.40)	46.19(−2.43)
Claude-Sonnet-4.6	16.16	7.32	37.98(+21.82)	23.62(+16.30)	90.48(+74.32)	45.87(+38.55)	89.37(+73.20)	41.90(+34.58)
Gemini-3.1-Pro	35.91	18.23	42.13(+6.22)	23.62(+5.39)	92.22(+56.31)	52.22(+33.99)	93.17(+57.26)	52.38(+34.15)
DeepSeek-V3.2	31.63	15.61	38.12(+6.49)	18.92(+3.31)	88.25(+56.62)	41.27(+25.66)	90.00(+58.37)	42.38(+26.77)
Kimi-K2.5	32.60	13.54	60.22(+27.62)	28.73(+15.19)	89.84(+57.24)	42.38(+28.85)	90.16(+57.56)	43.49(+29.96)
Qwen3.5-397B	20.03	9.81	37.29(+17.27)	20.58(+10.77)	89.68(+69.65)	46.83(+37.02)	90.16(+70.13)	45.87(+36.07)
Qwen3-235B	28.73	6.91	26.93(−1.80)	7.46(+0.55)	87.14(+58.41)	41.90(+35.00)	89.52(+60.79)	43.81(+36.90)
Qwen3-30B	41.57	16.16	43.37(+1.80)	15.75(−0.41)	86.83(+45.25)	41.11(+24.95)	86.67(+45.09)	40.32(+24.16)
MiniMax-M2.5	15.88	6.49	28.31(+12.43)	13.54(+7.04)	88.89(+73.00)	45.08(+38.59)	88.89(+73.00)	43.65(+37.16)
Llama-4-Scout	23.34	8.15	20.99(−2.35)	7.87(−0.28)	78.57(+55.23)	32.38(+24.23)	80.63(+57.29)	31.90(+23.76)

negative gain is consistent with both factors: when retrieval is well-matched, it provides modest benefit; when it is mismatched, it can degrade generation. The residual compatibility failures are therefore more likely attributable to limitations in version-API co-generation ability [25, 39, 41, 78] than to ignorance of the API surface, which needs further design that is outside this paper’s scope.

7 Discussion

LLMs routinely introduce version annotations that are vulnerable, incompatible, or both, driven by a shared bias rather than individual model defects. The problem resists prompt-level remediation but is addressable through external tooling. This section draws practical implications for developers, LLM providers, and the research community.

7.1 The Problem Is Ecosystem-Level, Not Model-Level

The cross-model convergence documented in §5.2.2 is the most structurally significant finding of this study. `django==6.0.1`, `requests==2.31.0`, and `flask==3.1.2` appear among the most-specified vulnerable pairs for at least nine of the ten evaluated models, with counts from closed-source and open-source families alike confirming the pattern as illustrated in Figure 6. No individual model accounts for this pattern, which reflects a shared distribution in the training corpora.

The mechanism follows directly from the temporal lag established in §5.1.4: versions released years before a model’s knowledge cutoff accumulate far greater training signal than recent ones, and those high-salience versions are precisely the ones that have had time to accumulate unpatched CVEs. The median lag ranges from 9 to 31 months across all ten models. This salience-vulnerability coupling is a structural consequence of web-crawled training data, not a correctable defect in any individual model, which is why practitioners cannot reduce exposure by switching models. Risk must be addressed at the point of version resolution.

7.2 Implications for Developers

LLM-specified versions require independent security validation before adoption. As documented in §5.2.1, over a third to more than half of all inline-prompting tasks contain a version with a known CVE, and the majority of those vulnerable versions are rated *Critical* or *High*. Accepting LLM-generated version annotations without review is sampling from a distribution heavily skewed toward high-severity vulnerabilities. Standard practice should route LLM-specified versions through an SCA tool or OSV query before committing them to a dependency manifest.

The prompting mode is itself a risk variable, and neither mode is universally safe. Switching from inline to explicit mode reduces the version specification rate by up to 88.73 percentage points as in §5.1.1. This substantially lowers the surface area for version-level vulnerability,

since unspecified dependencies cannot carry known CVEs. However, from the results in Table 8, explicit mode introduces a different risk: those unspecified dependencies are resolved to the latest available release at install time, which collapses compatibility under older Python environments for most models. In production codebases, explicit mode needs to be carefully used because it may omit dependency versions. In exploratory contexts, inline mode output should be treated as an audit signal rather than a trusted declaration. Overall, developers should include package security audit tools in the coding pipeline *e.g.*, `pip-audit` [50], to improve software supply chain security in the LLM-assisted coding.

Minor version substitution improves compatibility but not security. As discussed in §5.2.4, 95.11% of vulnerable (ℓ, v) pairs have no CVE-free release in the same major version branch, meaning that a patch-level increment is structurally insufficient to escape vulnerability exposure. On the compatibility dimension, the diagnosis experiment in §6.2 shows that neighboring-version search does recover installation failures in 17%–55% of cases, but these recovered versions are not guaranteed to be CVE-free. Developers should therefore anticipate a cross-major migration or a security-aware resolver rather than a simple patch-level increment.

7.3 Implications for LLM Providers

The version-selection problem is a grounding gap, not a safety alignment gap, and cannot be closed through prompting alone. As shown in Table 10, the `abl-instruct` condition reduces ρ_U by at most 2.2 percentage points across all ten models. The CVE disclosure analysis in §5.2.3 explains why: 72%–91% of the CVEs carried by LLM-specified versions were already publicly disclosed before each model’s training cutoff, so the model is not unaware of the vulnerability landscape. The failure is in mapping a version identifier to its CVE status at generation time. Addressing this requires changes to the training data or the inference architecture, not to the prompt. On the training side, the version-salience bias in §5.1.4 points to a concrete intervention: flagging (ℓ, v) pairs that appear frequently in training text while carrying an active OSV record, then de-weighting or annotating them with a security signal. The fact that `abl-version` reduces ρ_U by 20–36 percentage points relative to baseline confirms that models can follow externally provided version constraints; the gap is in spontaneous selection.

Inference-time information injecting can help. No training-time intervention can cover vulnerability disclosures that post-date the cutoff. The 9–31 month median lag will persist as long as models rely on static snapshots. Architectures that query a live vulnerability feed at inference time address this limitation directly. Separately, current code generation benchmarks measure logical correctness and functional pass rates but do not evaluate whether the dependency versions a model selects are safe, valid, or installable. The measurement framework developed in this study provides a reusable template for such evaluation, and incorporating version-annotation quality into routine benchmarks would create direct incentives for providers to close the grounding gap.

Community disclosure has produced initial acknowledgment from model providers. We disclosed the observed version-selection behavior to the community of all 10 evaluated models and to AI coding assistant providers, including GitHub Copilot and Cursor. Several parties have responded. OpenAI’s support channel acknowledged that *“right now there’s no built-in CVE check when suggesting versions, so this gap shows up across models...”*¹ and indicated that the feedback would be forwarded to the product team. Alibaba Cloud and Moonshot AI similarly confirmed receipt of the findings through their respective support channels. These responses corroborate that the version-selection gap reflects a recognized but currently unaddressed behavior in LLMs.

¹<https://community.openai.com/t/1378999>

8 Threats to Validity

In this section, we report the internal and external validity problems of this study and discuss the measures we take to relieve their impact.

8.1 Internal Validity

Construct operationalization. Static compatibility is operationalized through installation success and `ty` static type checking, neither of which is equivalent to full runtime correctness. `ty` does not detect semantic errors in generated logic or behaviors not captured by type stubs. We mitigate this by complementing Stage (5) with BigCodeBench dynamic execution as designed in §5.3.3. The Sankey analysis in §5.3.4 quantifies the residual gap between the two signals. Vulnerability exposure is operationalized as CVE presence in the OSV index, which is a conservative upper bound on exploitability rather than a precise risk measure. Exploitability assessment would require per-task taint analysis, which is beyond this study's scope.

Static type checker bias. τ_C depends on `ty`, which may both under- and over-report incompatibility. The Sankey analysis in §5.3.4 bounds the false positive rate at 8.15%: that fraction of `ty`-incompatible tasks still pass BigCodeBench execution. Libraries lacking type stubs are largely invisible to `ty`, causing τ_C to overestimate compatibility for poorly-typed packages. The BigCodeBench dynamic evaluation mitigates both concerns, as execution evidence is not subject to stub coverage constraints.

Sampling variance. We generate one output per (t, m, p) triple using provider-default temperature settings, reflecting the deployment conditions most developers encounter. Characterizing within-model variance in version annotation is outside the scope of this study. Future work could sample multiple outputs per triple to bound this variability.

Prompting mode coverage. The two evaluated modes cover the most common developer interaction patterns. Agentic workflows, multi-turn refinement, and IDE-level system prompts may exhibit different version-annotation behavior, and we make no claims about these settings. Our study establishes a baseline for the most frequent interaction modes. Thus, characterizing the full space of prompting contexts is a direction for future work.

8.2 External Validity

Ecosystem scope. This study is scoped to Python and PyPI. Version-pinning risks plausibly exist in other ecosystems, *e.g.*, npm, Maven, Cargo, but vulnerability exposure rates, compatibility failure patterns, and model behavior may differ substantially. We make no generalizability claims beyond Python, and it can be a good direction for future work.

Dataset representativeness. The PinTrace dataset employs TPL-balanced sampling, but the underlying question distribution still favors high-activity libraries and pre-LLM-adoption periods, *i.e.*, 313 tasks from 2020 versus 40 from 2025. Niche libraries and post-2024 development contexts may exhibit different version-annotation behavior than what we observe. This is a limitation caused by the huge reduction to 10% of remaining questions on Stack Overflow itself [61] as discussed in section §4.1, which may need new dataset sources to overcome in future work.

Model snapshot. The 10 evaluated models represent a snapshot of the model landscape as of early 2026. Absolute vulnerability exposure rates and model rankings may change as models are updated. To alleviate this, we included the Qwen3 and Qwen3.5 model families in the experiments, and the findings are consistent. The structural findings, including the temporal lag, the grounding gap, and the ineffectiveness of language-level safety instructions, are more likely to generalize, as they reflect overall characteristics of LLMs instead of a single one.

Table 12. Comparison of related studies along key dimensions. ✓: the study addresses this dimension, ✗: the dimension is outside the study’s scope.

Dimension	Fu <i>et al.</i> [8]	Spracklen <i>et al.</i> [59]	Latendresse <i>et al.</i> [32]	Wang <i>et al.</i> [66]	Wu <i>et al.</i> [71]	This work
Logic security	✓	✗	✗	✗	✗	✗
Package existence	✗	✓	✓	✗	✗	✓
Version safety	✗	✗	✗	✗	✗	✓
Version compatibility	✗	✗	✗	✓	✓	✓
Spontaneous selection	✗	✓	✓	✗	✗	✓

9 Related Work

9.1 LLM Code Quality and Package Selection

LLM-generated code introduces risks successively: whether the generated *logic* is secure, whether the *packages* it imports exist, and whether the *versions* of those packages are safe and installable. Prior work has addressed the first two levels extensively, and this paper addresses the third.

Logic-level security. Early studies established that GitHub Copilot produces snippets containing CWE-classified weaknesses at non-trivial rates, spanning SQL injection, hard-coded credentials, and cryptographic misuse [8]. Subsequent large-scale analyses confirmed that a significant fraction of generated snippets trigger CWE-classified weaknesses [5, 10, 77], and Sajadi *et al.* showed that security guidance is frequently absent from LLM responses to Stack Overflow-style questions [53]. On the mitigation side, proposed interventions include instruction tuning on security-aware datasets [14, 17], constrained decoding guided by static analysis [7, 34, 35], and RAG-augmented prompting with security guidelines [37]. Wang *et al.* characterized security risks in LLM-generated code found in public GitHub repositories [65]. None of these mitigations addresses the version dimension: even logically secure code can expose its runtime environment to catalogued vulnerabilities through the dependency versions it pins.

Package existence and recommendation. Package confusion is a known software supply chain threat [43], and LLM interfaces have made it more consequential [76]. Spracklen *et al.* conducted the most comprehensive study of package hallucination, evaluating 16 LLMs on 576,000 generated samples and finding hallucinated package names at rates from 5.2% for commercial models to 21.7% for open-source models [59]. Latendresse *et al.* examined ChatGPT’s library recommendations and showed that models frequently recommend outdated or contextually inappropriate packages [32, 33]. Sipio *et al.* documented a popularity bias in LLM-based TPL recommendations [56]. These studies establish that LLM package selection is systematically biased and occasionally hallucinatory, but none examines *version* specifications. A correctly named, non-hallucinated package can still expose users to risk if the specified version carries a known CVE. Our work fills this gap: assuming the package name is correct, do LLMs choose a version that is safe and installable?

9.2 Version Awareness in LLM-Assisted Development

A separate line of work examines whether LLMs can correctly reason about specific library versions. Two research directions have emerged: one studies whether generated code is compatible with a *given* version, the other studies whether generated code can be adapted when APIs evolve. Both directions presuppose that the version is externally provided or already known. Neither asks whether the LLM’s *spontaneous* version choice is safe and installable.

Compatibility with a given version. Wang *et al.* evaluated LLM-based code completion on tasks involving deprecated APIs, finding that models recommend deprecated usage even when safer alternatives exist in more recent versions, with training data recency as the primary driver [66]. Wu *et al.* introduced VersiCode, a benchmark spanning 300 libraries and 2,207 versions, and showed that even GPT-4o scores more than 50 points lower than on standard benchmarks when the task requires version-specific code completion [71]. Islah *et al.* introduced GitChameleon, an execution-based

benchmark of 116 Python problems each conditioned on a specific library version, finding GPT-4o achieves only 39.9% pass@10 on version-correct generation [23]. Misra *et al.* extended this with GitChameleon 2.0 that has 328 problems with executable unit tests, finding that state-of-the-art systems achieve only 48–51% success rates on version-conditioned generation [42]. Kuhar *et al.* proposed LibEvolutionEval and demonstrated that model accuracy in version-specific generation degrades for recently released versions with limited training coverage [27, 28].

Adaptation to API evolution. Wu *et al.* showed that environment-unaware code generation produces non-executable code at high rates when the deployment environment diverges from training-time assumptions [70]. Liu *et al.* constructed CodeUpdateArena to benchmark knowledge editing for API changes, evaluating how well models adapt code when an API evolves between versions [39]. Work on compatibility failure analysis at the ecosystem level [25, 41, 67, 78] further documents the scale of API-breaking changes across library releases.

Positioning. These benchmarks address a logically downstream question: *given a specified version, can the LLM generate compatible code?* Our work addresses the logically prior question: *do LLMs choose secure and compatible versions in the first place?* Version selection precedes code generation, and both must be answered to fully characterize the risk profile of LLM-assisted dependency management. Unlike benchmark-centric evaluations that present models with explicit version constraints on curated tasks, our study measures spontaneous version-annotation behavior on PinTrace, directly capturing what developers receive from LLMs under normal usage conditions. Table 12 summarizes how our work differs from the most closely related studies.

10 Conclusion

LLMs are now a primary interface through which developers specify software dependencies, yet the security and compatibility of the version choices they produce have received no systematic study. We evaluated ten LLMs on PinTrace and identified three patterns that together characterize version selection as a previously overlooked risk surface.

First, version-annotation behavior is governed by format affordance rather than consistent engineering intent. Inline prompting elicits specification rates up to 95.18%, while explicit mode produces as few as 6.45%. Across both modes, version choices concentrate on a narrow band of popular releases that lags each model’s knowledge cutoff by 9 to 31 months.

Second, the specified versions are systematically vulnerable. Under inline prompting, 36.70%–55.70% of tasks carry at least one CVE, with 62.75%–74.51% of vulnerable versions rated *Critical* or *High* severity. The same small set of vulnerable TPL and version pairs appears across all ten model families, driven by a shared training-signal bias rather than individual model error.

Third, the specified versions are frequently incompatible. Static compatibility rates fall to 19.70%–63.20% under inline prompting, and dynamic BigCodeBench pass rates collapse to 6.49%–48.62%. A controlled neighboring-version experiment confirms that 17%–55% of installation failures are attributable to version selection rather than to code-logic defects.

In the near term, practitioners should treat LLM-specified versions as unverified suggestions and route them through validation before committing to a dependency manifest. The broader implication is that the field must extend its trust model for LLM-generated code beyond logical correctness to encompass the supply-chain dimensions that dependency versions reach into. We disclosed these findings to all evaluated model providers and to major AI coding assistant platforms. Several parties confirmed the issue and passed the information to the tech team for further improvements. As AI-assisted development increasingly generates dependency annotations at scale, version selection will account for a growing share of the attack surface that developers inadvertently accept.

References

- [1] Rahaf Alkhadra, Joud Abuzaid, Mariam AlShammari, and Nazeeruddin Mohammad. 2021. Solar winds hack: In-depth analysis and countermeasures. In *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, IEEE, Kharagpur, India, 1–7. doi:10.1109/ICCCNT51525.2021.9579611
- [2] Astral. 2024. uv: An Extremely Fast Python Package Installer and Resolver. <https://docs.astral.sh/uv/>.
- [3] Astral. 2025. ty: A Fast Type Checker and Language Server for Python. <https://docs.astral.sh/ty/>.
- [4] Bndr and contributors. 2024. pipreqs: Package-to-Import Name Mapping. <https://github.com/bndr/pipreqs/blob/master/pipreqs/mapping>. Accessed: February 2026.
- [5] Shih-Chieh Dai, Jun Xu, and Guan hong Tao. 2026. Rethinking the Evaluation of Secure Code Generation. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. doi:10.48550/arXiv.2503.15554
- [6] Andreas Dann, Ben Hermann, and Eric Bodden. 2023. UpCy: Safely Updating Outdated Dependencies. In *Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE '23)*. IEEE Press, 233–244. doi:10.1109/ICSE48619.2023.00031
- [7] Yanjun Fu, Ethan Baker, Yu Ding, and Yizheng Chen. 2024. Constrained Decoding for Secure Code Generation. arXiv:2405.00218 doi:10.48550/arXiv.2405.00218
- [8] Yujia Fu, Peng Liang, Amjed Tahir, et al. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–34. doi:10.1145/3716848
- [9] Kai Gao, Runzhi He, Bing Xie, and Minghui Zhou. 2024. Characterizing Deep Learning Package Supply Chains in PyPI: Domains, Clusters, and Disengagement. *ACM Trans. Softw. Eng. Methodol.* 33, 4, Article 97 (April 2024), 27 pages. doi:10.1145/3640336
- [10] Ruofan Gao, Amjed Tahir, Peng Liang, Teo Susnjak, and Foutse Khomh. 2025. A Survey of Bugs in AI-Generated Code. arXiv:2512.05239 doi:10.48550/arXiv.2512.05239
- [11] GitHub. 2025. GitHub Copilot: AI Code Generation Statistics. <https://github.blog/news-insights/research/>.
- [12] GitHub. 2025. Octoverse 2025: A New Developer Joins GitHub Every Second as AI Leads TypeScript to #1. <https://github.blog/news-insights/octoverse/>.
- [13] Google. 2021. OSV: Open Source Vulnerabilities. <https://osv.dev>.
- [14] Mohammad Saqib Hasan, Saikat Chakraborty, Santu Karmaker, and Niranjana Balasubramanian. 2025. Teaching an old LLM secure coding: Localized preference optimization on distilled preferences. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 26039–26057.
- [15] Hao He, Bogdan Vasilescu, and Christian Kästner. 2025. Pinning Is Futile: You Need More Than Local Dependency Versioning to Defend against Supply Chain Attacks. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE013 (June 2025), 24 pages. doi:10.1145/3715728
- [16] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003
- [17] Jingxuan He, Mark Vero, Gabriela Krasnopolska, and Martin Vechev. 2024. Instruction Tuning for Secure Code Generation. In *International Conference on Machine Learning*. PMLR, 18043–18062.
- [18] Runzhi He, Hao He, Yuxia Zhang, and Minghui Zhou. 2023. Automating Dependency Updates in Practice: An Exploratory Study on GitHub Dependabot. *IEEE Trans. Softw. Eng.* 49, 8 (Aug. 2023), 4004–4022. doi:10.1109/TSE.2023.3278129
- [19] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [20] Jinchang Hu, Lyuye Zhang, Chengwei Liu, Sen Yang, Song Huang, and Yang Liu. 2024. Empirical Analysis of Vulnerabilities Life Cycle in Golang Ecosystem. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE, Lisbon, Portugal, 2618–2630. doi:10.1145/3597503.363923
- [21] Nasif Intiaz, Seaver Thorn, and Laurie Williams. 2021. A Comparative Study of Vulnerability Reporting by Software Composition Analysis Tools. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, Bari, Italy, Article 5. doi:10.1145/3475716.3475769
- [22] Nasif Intiaz and Laurie Williams. 2023. Are Your Dependencies Code Reviewed?: Measuring Code Review Coverage in Dependency Updates. *IEEE Transactions on Software Engineering* 49, 11 (Nov. 2023), 4932–4945. doi:10.1109/TSE.2023.3319509
- [23] Nizar Islah, Justine Gehring, Diganta Misra, Eilif Muller, Irina Rish, Terry Yue Zhuo, and Massimo Caccia. 2024. GitChameleon: Unmasking the Version-Switching Capabilities of Code Generation Models. arXiv:2411.05830 [cs.SE] <https://arxiv.org/abs/2411.05830>
- [24] Damien Jaime, Pascal Poizat, Joyce El Haddad, and Thomas Dague. 2024. Balancing the Quality and Cost of Updating Dependencies. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*

- (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1834–1845. doi:10.1145/3691620.3695595
- [25] Nihal Jain, Robert Kwiatkowski, Baishakhi Ray, Murali Krishna Ramanathan, and Varun Kumar. 2025. On Mitigating Code LLM Hallucinations with API Documentation. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 237–248. doi:10.1109/ICSE-SEIP66354.2025.00027
 - [26] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 35, 2, Article 58 (Jan. 2026), 72 pages. doi:10.1145/3747588
 - [27] Sachit Kuhar, Wasi Uddin Ahmad, Zijian Wang, Nihal Jain, Haifeng Qian, Baishakhi Ray, Murali Krishna Ramanathan, Xiaofei Ma, and Anoop Deoras. 2025. LibEvolutionEval: A Benchmark and Study for Version-Specific Code Generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 6826–6840. doi:10.18653/v1/2025.naacl-long.348
 - [28] Sachit Kuhar, Wasi Uddin Ahmad, Zijian Wang, Nihal Jain, Haifeng Qian, Baishakhi Ray, Murali Krishna Ramanathan, Xiaofei Ma, and Anoop Deoras. 2025. LibEvolutionEval: A Benchmark and Study for Version-Specific Code Generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 6826–6840. doi:10.18653/v1/2025.naacl-long.348
 - [29] Raula Gaikovina Kula, Daniel M. German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? *Empirical Softw. Engg.* 23, 1 (Feb. 2018), 384–417. doi:10.1007/s10664-017-9521-5
 - [30] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 1509–1526. doi:10.1109/SP46215.2023.10179304
 - [31] Jasmine Latendresse, Naoures Day, SayedHassan Khatoonabadi, and Emad Shihab. 2025. The Software Librarian: Python Package Insights for Copilot. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Companion Proceedings*. 17–20. doi:10.1109/ICSE-Companion66252.2025.00014
 - [32] Jasmine Latendresse, SayedHassan Khatoonabadi, Ahmad Abdellatif, and Emad Shihab. 2024. Is ChatGPT a Good Software Librarian? An Exploratory Study on the Use of ChatGPT for Software Library Recommendations. arXiv:2408.05128 doi:10.48550/arXiv.2408.05128
 - [33] Jasmine Latendresse, SayedHassan Khatoonabadi, and Emad Shihab. 2025. How Robust Are LLM-Generated Library Imports? An Empirical Study Using Stack Overflow. arXiv:2507.10818 doi:10.48550/arXiv.2507.10818
 - [34] Dong Li, Shanfu Shu, Meng Yan, et al. 2025. Improving Co-Decoding Based Security Hardening of Code LLMs Leveraging Knowledge Distillation. *IEEE Transactions on Software Engineering* 51, 9 (2025), 2634–2650. doi:10.1109/TSE.2025.3591791
 - [35] Dong Li, Meng Yan, Yaosheng Zhang, et al. 2024. CoSec: On-the-Fly Security Hardening of Code LLMs via Supervised Co-Decoding. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1428–1439. doi:10.1145/3650212.3680371
 - [36] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025. FEA-Bench: A Benchmark for Evaluating Repository-Level Code Generation for Feature Implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 17160–17176. doi:10.18653/v1/2025.acl-long.839
 - [37] Bo Lin, Shangwen Wang, Yihao Qin, Liqian Chen, and Xiaoguang Mao. 2025. Give LLMs a Security Course: Securing Retrieval-Augmented Code Generation via Knowledge Injection. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (Taipei, Taiwan) (CCS '25)*. Association for Computing Machinery, New York, NY, USA, 3356–3370. doi:10.1145/3719027.3765049
 - [38] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the npm Ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. IEEE, Pittsburgh, PA, USA, 672–684. doi:10.1145/3510003.3510142
 - [39] Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. 2025. CodeUpdateArena: Benchmarking Knowledge Editing on API Updates. arXiv:2407.06249 doi:10.48550/arXiv.2407.06249
 - [40] Tarek Mahmud, Meiru Che, and Guowei Yang. 2023. Detecting Android API Compatibility Issues With API Differences. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3857–3871. doi:10.1109/TSE.2023.3274153
 - [41] Tarek Mahmud, Bin Duan, Meiru Che, Awatif Yasmin, Anne H. H. Ngu, and Guowei Yang. 2026. Automated Update of Android Deprecated API Usages With Large Language Models. *IEEE Transactions on Software Engineering* 52, 1 (2026), 70–85. doi:10.1109/TSE.2025.3627897
 - [42] Diganta Misra, Nizar Islah, Victor May, Brice Rauby, Zihan Wang, Justine Gehring, Antonio Orvieto, Muawiz Sajjad Chaudhary, Elif B. Muller, Irina Rish, Samira Ebrahimi Kahou, and Massimo Caccia. 2025. GitChameleon 2.0: Evaluating AI Code Generation Against Python Library Version Incompatibilities. In *NeurIPS 2025 Fourth Workshop on Deep*

- Learning for Code*. <https://openreview.net/forum?id=wtqdcVfJUN>
- [43] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond typosquatting: an in-depth look at package confusion. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 193, 18 pages.
 - [44] NTIA. 2019. Framing Software Component Transparency: Establishing a Common Software Bill of Material (SBOM). https://ntia.gov/files/ntia/publications/framingsbom_20191112.pdf
 - [45] NVD. 2021. CVE-2021-44228 Detail: Apache Log4j2 Remote Code Execution Vulnerability. <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>.
 - [46] OpenAI. 2022. Tiktoken. <https://github.com/openai/tiktoken>. Accessed: February 2026.
 - [47] Ivan Pashchenko, Duc-Ly Vu, and Fabio Massacci. 2020. A Qualitative Study of Dependency Management and Its Security Implications. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) (CCS '20). Association for Computing Machinery, New York, NY, USA, 1513–1531. doi:10.1145/3372297.3417232
 - [48] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2020. Detection, Assessment and Mitigation of Vulnerabilities in Open Source Dependencies. *Empirical Software Engineering* 25, 5 (2020), 3175–3215. doi:10.1007/s10664-020-09830-x
 - [49] Piotr Przymus and Thomas Durieux. 2025. Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. In *Proceedings of the 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE, Ottawa, ON, Canada, 91–102. doi:10.1109/MSR66628.2025.00026
 - [50] pypa. 2021. pip-audit. <https://github.com/pypa/pip-audit>. Accessed: February 2026.
 - [51] Python. 2002. The Python Package Index (PyPI). <https://pypi.org/>. Accessed: February 2026.
 - [52] Python. 2023. Python Release Python 3.12. <https://www.python.org/downloads/release/python-3120/>. Accessed: February 2026.
 - [53] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do LLMs Consider Security? An Empirical Study on Responses to Programming Questions. *Empirical Software Engineering* 30, 4 (2025), 101. doi:10.1007/s10664-025-10658-6
 - [54] Yijun Shen, Xiang Gao, Hailong Sun, and Yu Guo. 2025. Understanding Vulnerabilities in Software Supply Chains. *Empirical Software Engineering* 30, 1 (2025), 20. doi:10.1007/s10664-024-10581-2
 - [55] Yijun Shen, Xiang Gao, Hailong Sun, and Yu Guo. 2025. Understanding vulnerabilities in software supply chains. *Empirical Software Engineering* 30, 1 (2025), 20. doi:10.1007/s10664-024-10581-2
 - [56] Claudio Di Sipio, Juri Di Rocco, Davide Di Ruscio, and Vladyslav Bulhakov. 2025. Addressing Popularity Bias in Third-Party Library Recommendations Using LLMs . In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*. IEEE Computer Society, Los Alamitos, CA, USA, 33–40. doi:10.1109/SANER-C66551.2025.00012
 - [57] snyk. 2021. SolarWinds Orion Security Breach: A Shift In The Software Supply Chain Paradigm. <https://snyk.io/blog/solarwinds-orion-security-breach-a-shift-in-the-software-supply-chain-paradigm/>
 - [58] Sonatype. 2026. 2026 Software Supply Chain Report. <https://www.sonatype.com/state-of-the-software-supply-chain/>.
 - [59] Joseph Spracklen, Raveen Wijewickrama, AHM Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs. In *Proceedings of the 34th USENIX Conference on Security Symposium* (Seattle, WA, USA) (SEC '25). USENIX Association, USA, Article 190, 20 pages.
 - [60] Stack Exchange, Inc. 2026. Stack Exchange Data Dump. <https://stackoverflow.com/help/data-dumps>. Snapshot: Jan 6, 2026.
 - [61] Stack Exchange, Inc. 2026. Stack Exchange Data Query. <https://data.stackexchange.com/stackoverflow/query/edit/1903717#resultSets>. Snapshot: April 15, 2026.
 - [62] Stack Overflow. 2025. 2025 Stack Overflow Developer Survey. <https://survey.stackoverflow.co/2025/>.
 - [63] Zeyu Sun, Jingzheng Wu, Xiang Ling, Yilin Wei, Tianyue Luo, and Yanjun Wu. 2025. Research on Key Technologies of SBOM in Software Supply Chain. *Journal of Software* 36, 6 (June 2025), 2604. doi:10.13328/j.cnki.jos.007308
 - [64] Timothée Thiéblemont. 2021. Griffe: Signatures for Entire Python Programs. <https://mkdocstrings.github.io/griffe/>.
 - [65] Bin Wang, Wenjie Yu, Yilu Zhong, et al. 2025. AI Code in the Wild: Measuring Security Risks and Ecosystem Shifts of AI-Generated Code in Modern Software. arXiv:2512.18567 doi:10.48550/arXiv.2512.18567
 - [66] Chong Wang, Kaifeng Huang, Jian Zhang, et al. 2025. LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-Based Code Completion. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. 885–897. doi:10.1109/ICSE55347.2025.00245
 - [67] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2025. LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-Based Code Completion. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 885–897. doi:10.1109/ICSE55347.2025.00245

- [68] Chengjie Wang, Jingzheng Wu, Hao Lyu, Xiang Ling, Tianyue Luo, Yanjun Wu, and Chen Zhao. 2026. A Large Scale Empirical Analysis on the Adherence Gap between Standards and Tools in SBOM. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2026). doi:10.1145/3788692 Just Accepted.
- [69] Shenao Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2025. Large Language Model Supply Chain: A Research Agenda. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 147 (May 2025), 46 pages. doi:10.1145/3708531
- [70] Tongtong Wu, Rongyi Chen, Wenjie Du, et al. 2026. Environment-Aware Code Generation: How Far Are We?. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. doi:10.48550/arXiv.2601.12262
- [71] Tongtong Wu, Weigang Wu, Xingyu Wang, et al. 2024. VersiCode: Towards Version-Controllable Code Generation. arXiv:2406.07411 doi:10.48550/arXiv.2406.07411
- [72] Yulun Wu, Zeliang Yu, Ming Wen, Qiang Li, Deqing Zou, and Hai Jin. 2023. Understanding the Threats of Upstream Vulnerabilities to Downstream Projects in the Maven Ecosystem. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) (ICSE '23). IEEE Press, 1046–1058. doi:10.1109/ICSE48619.2023.00095
- [73] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE037 (June 2025), 24 pages. doi:10.1145/3715754
- [74] Yifan Xia, Zichen Xie, Peiyu Liu, Kangjie Lu, Yan Liu, Wenhai Wang, and Shouling Ji. 2025. Beyond Static Pattern Matching? Rethinking Automatic Cryptographic API Misuse Detection in the Era of LLMs. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA006 (June 2025), 24 pages. doi:10.1145/3728875
- [75] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Shengcheng Yu, Weisong Sun, Yun Yang, and Zhenyu Chen. 2026. A survey on large language models for software engineering. *Science China Information Sciences* 69, 4 (2026), 141102.
- [76] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA022 (June 2025), 23 pages. doi:10.1145/3728894
- [77] Jianguo Zhao, Yuqiang Sun, Cheng Huang, et al. 2025. Towards Secure Code Generation with LLMs: A Study on Common Weakness Enumeration. *IEEE Transactions on Software Engineering* (2025), 1–16. doi:10.1109/TSE.2025.3619281
- [78] Tingwei Zhu, Zhongzhen Wen, Shangqing Liu, Yi Li, Tian Zhang, and Xin Xia. 2026. Assessing the Capability of LLMs for Deprecated API Usage Updating from Natural Language Descriptions. *ACM Trans. Softw. Eng. Methodol.* (April 2026). doi:10.1145/3808230 Just Accepted.
- [79] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen GONG, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=YrycTjllLO>
- [80] Terry Yue Zhuo, Junda He, Jiamou Sun, Zhenchang Xing, David Lo, John Grundy, and Xiaoning Du. 2026. Identifying and Mitigating API Misuse in Large Language Models. *IEEE Transactions on Software Engineering* 52, 3 (2026), 855–873. doi:10.1109/TSE.2026.3651566